Executive Post Graduate Certificate in Applied AI and Machine Learning

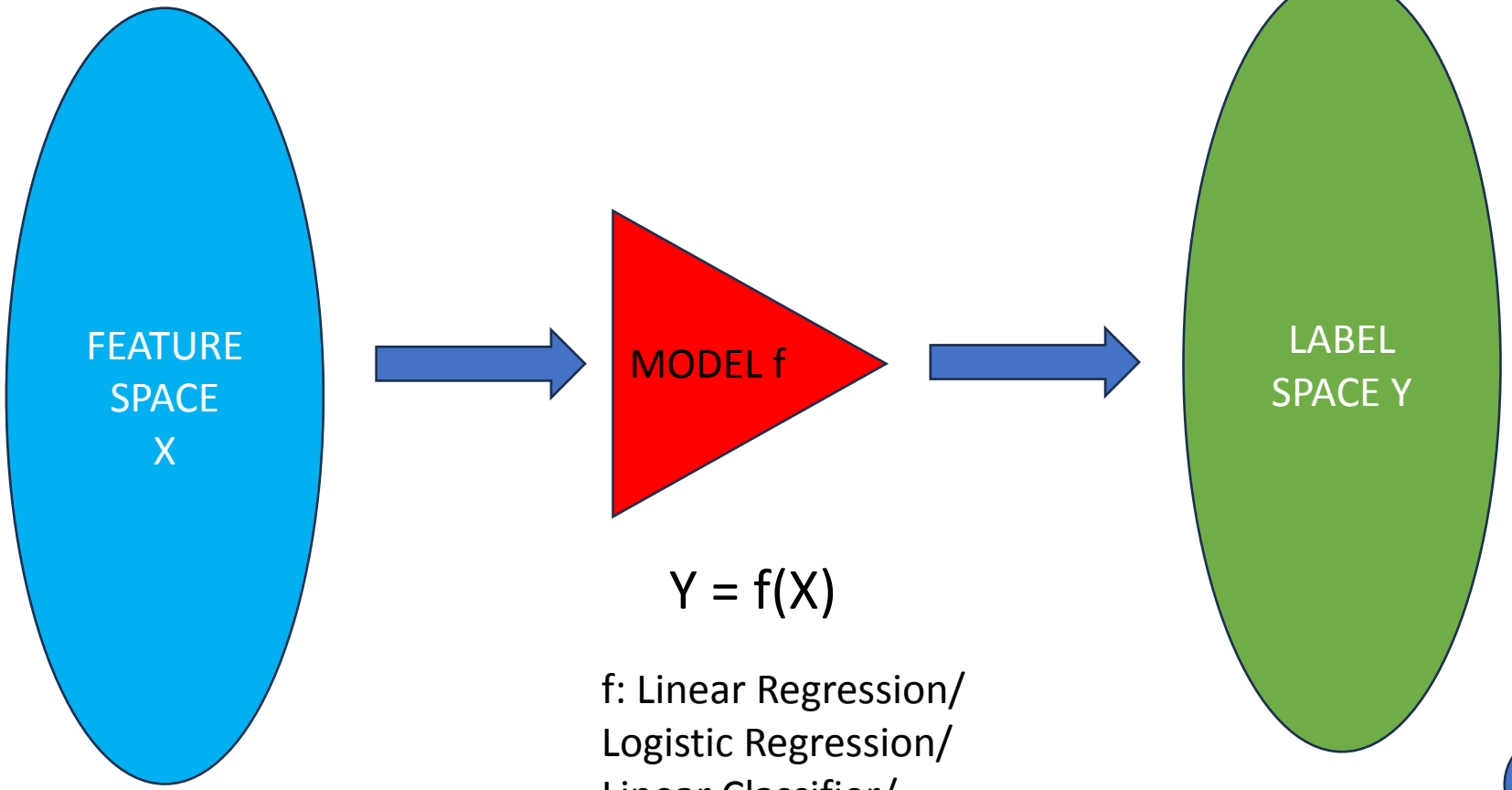
Module 1, Lecture 3

Linear Classifiers, Support Vector Machines, Bias-vs-Variance

Adway Mitra

Department of Artificial Intelligence, IIT Kharagpur

Supervised Learning



Y = f(X)

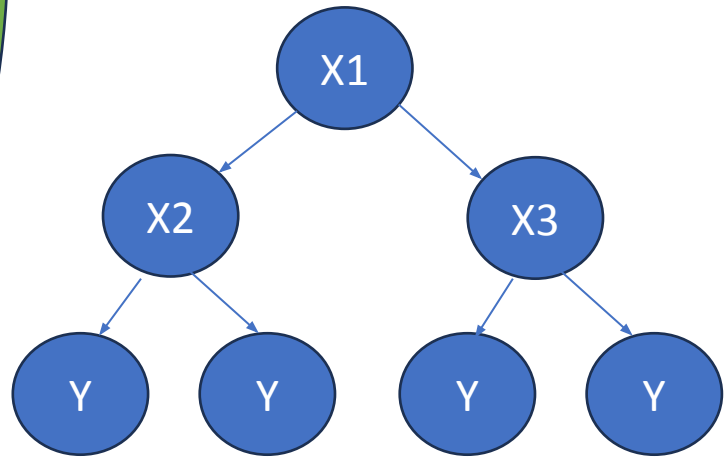
f: Linear Regression/ Logistic Regression/ Linear Classifier/ Decision Tree / Neural Network ……

Y = w0 + w1*X1 + w2*X2 + w3*X3

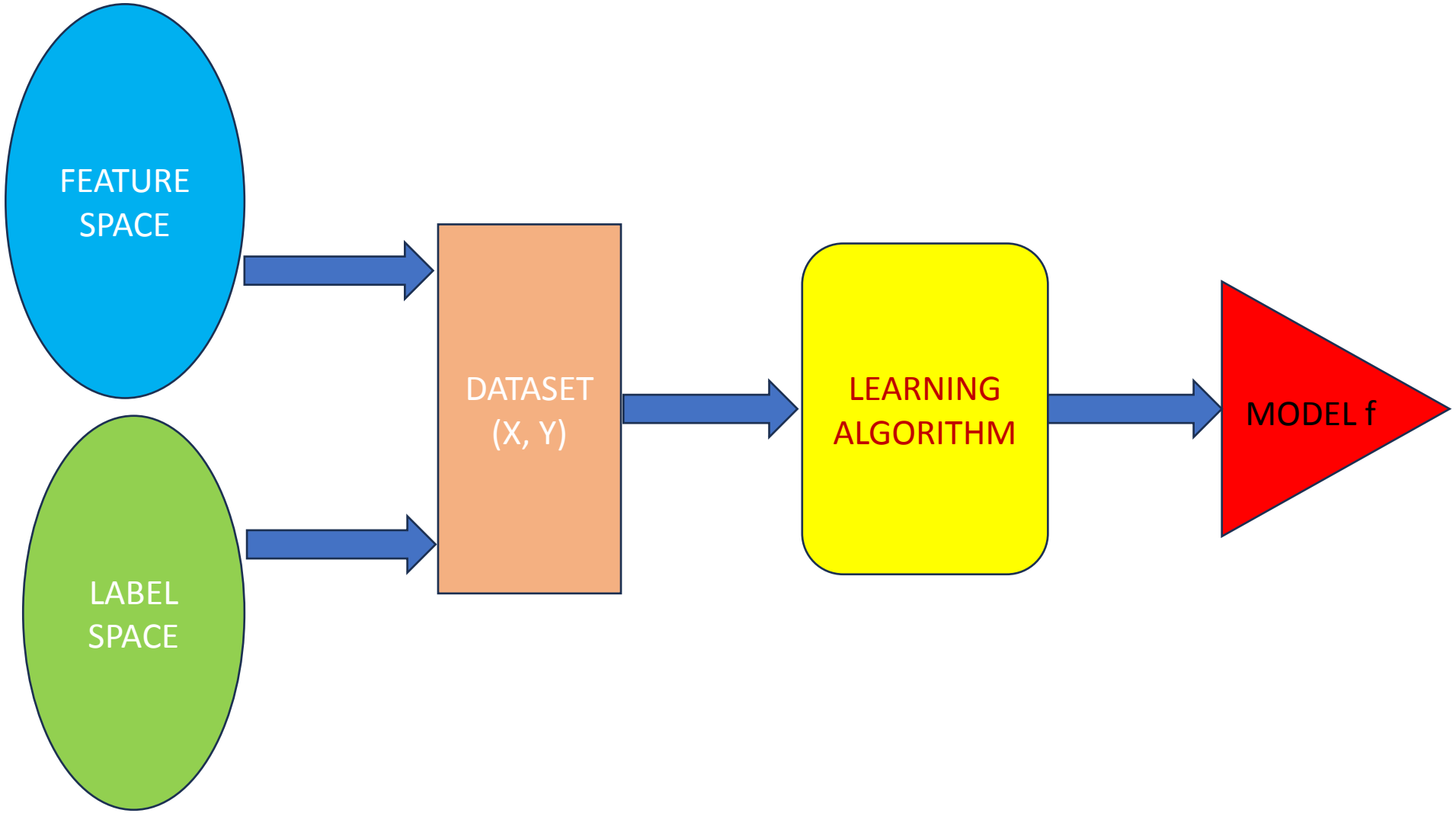
Y = w0 + w1*X1 + w2*X2*X2 + w3*X2*X3

Y = sign(w0 + w1*X1 + w2*X2 + w3*X3)

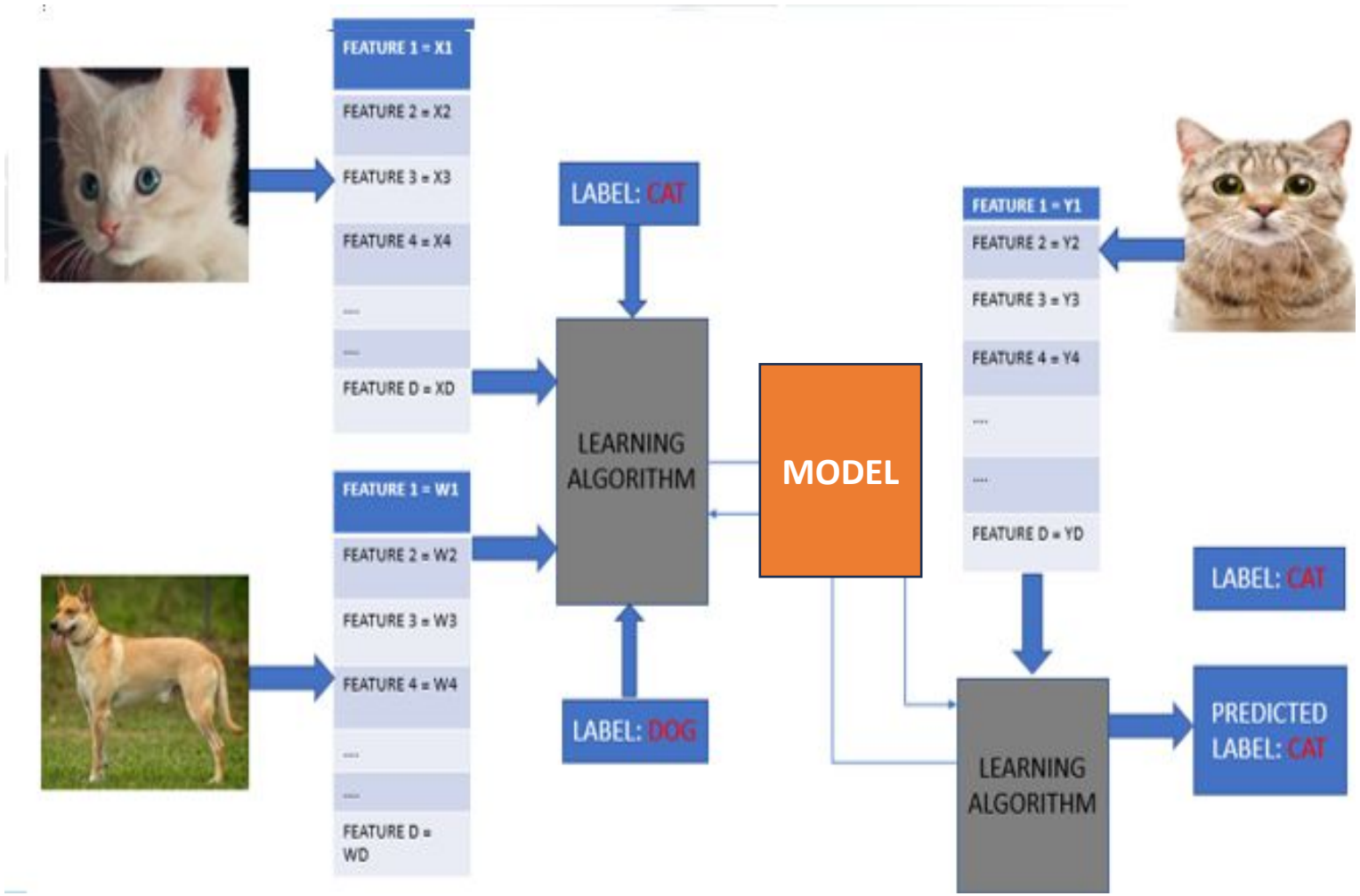
P(Y=1|X) = 1/(1+exp(w1*X1))



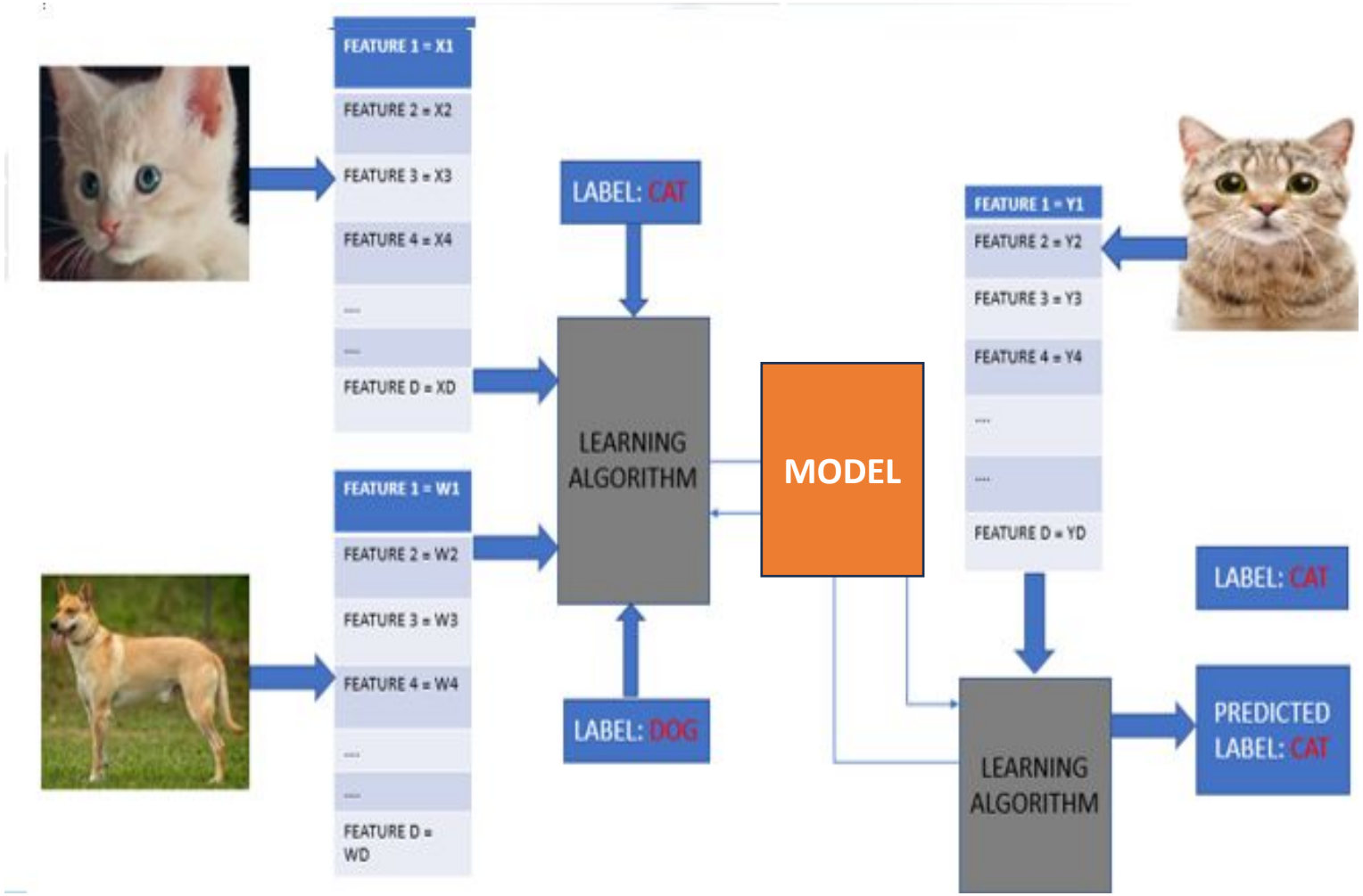
Supervised Learning



Supervised Learning



Supervised Learning

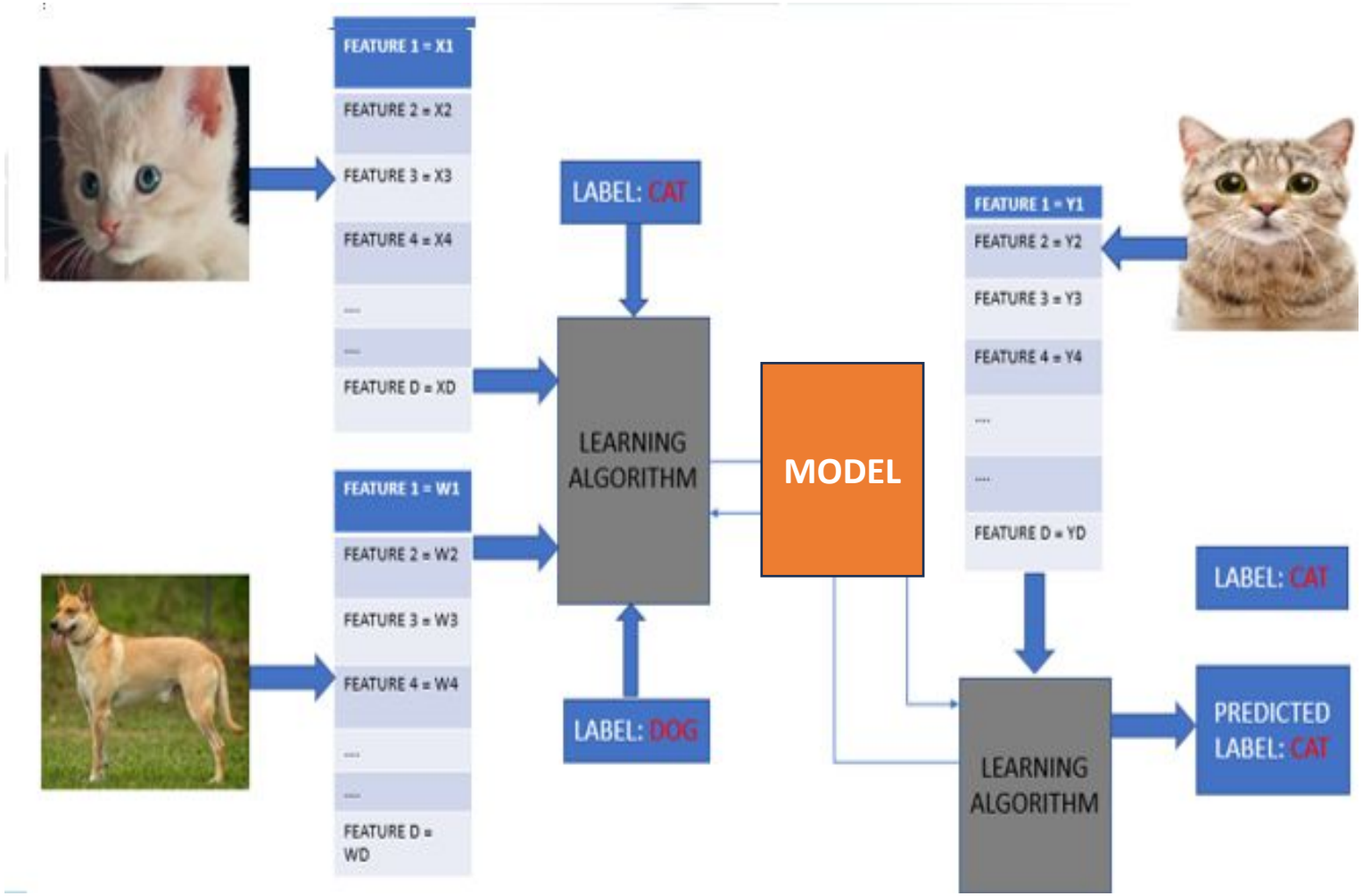


What are the features?

What is the model?

What is the learning algorithm?

Supervised Learning



What are the features?

- Any measurable attributes of the objects to be classified
- Numeric, categorical, ordinal ……

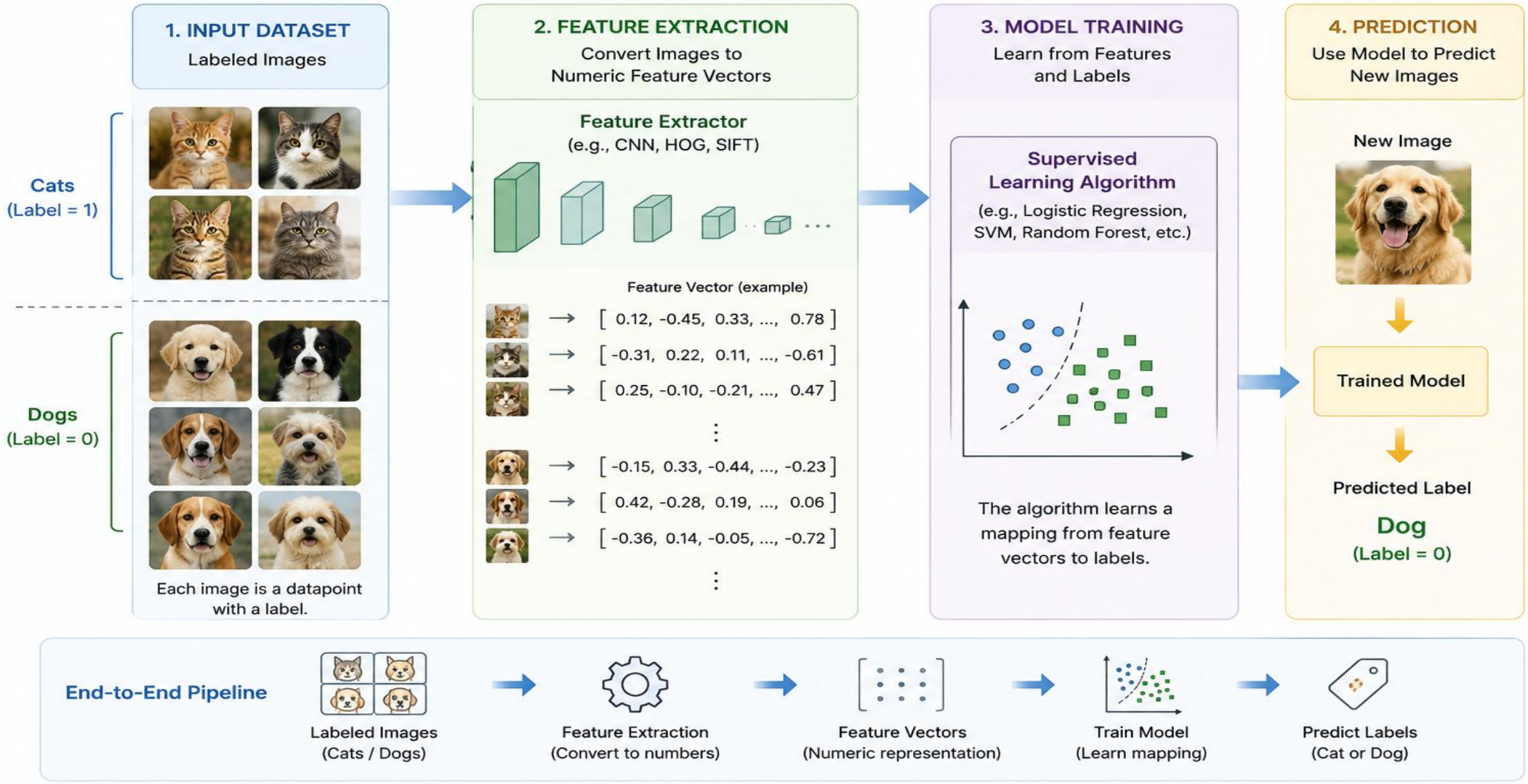
What is the model?

- An imagined relation between input and output, with some parameters!

What is the learning algorithm?

- Any algorithm that can estimate the parameters of the model, by minimizing a LOSS FUNCTION

Supervised Learning Pipeline

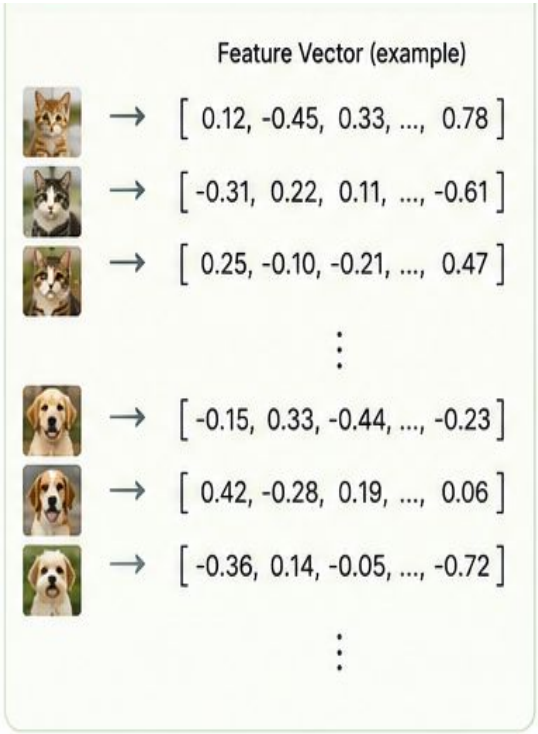


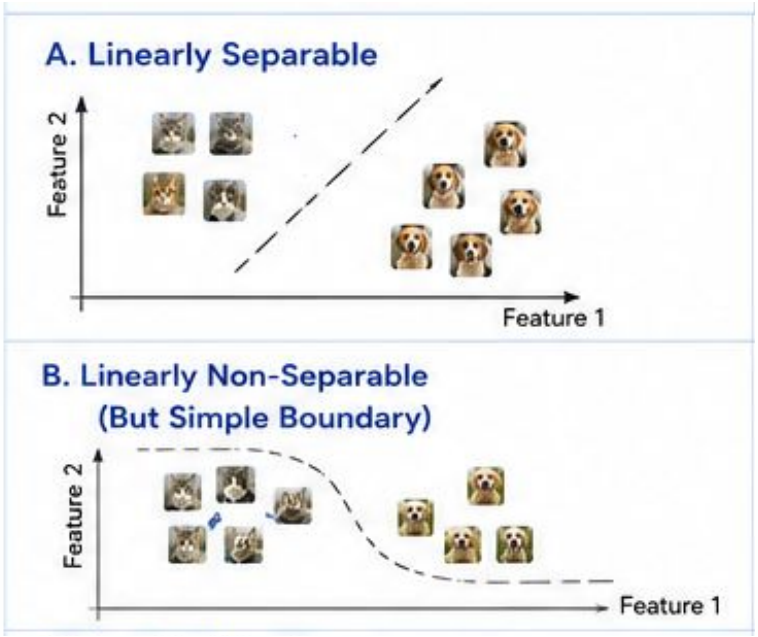
Concept of ML Model

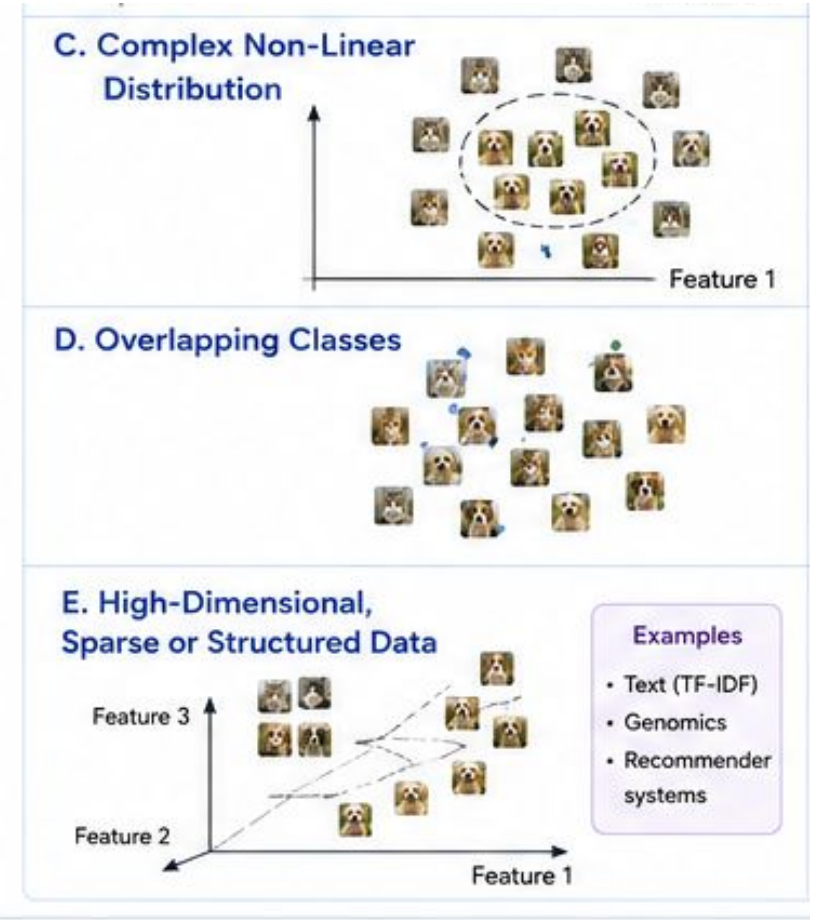
- There is no pre-defined mathematical relation between the features and the labels!
- We may imagine such a description for computational purposes
- **ML model: an imagined mathematical description of the data!**
- Label y = f(features x)!
    - simple linear model: y = 2x+3
    - More complex model: prob(y=1) = 1/(1+exp(-3x1+x2))

Some models are explicit, others are implicit (cannot be described by a parametric function)!

Features, Label and Model







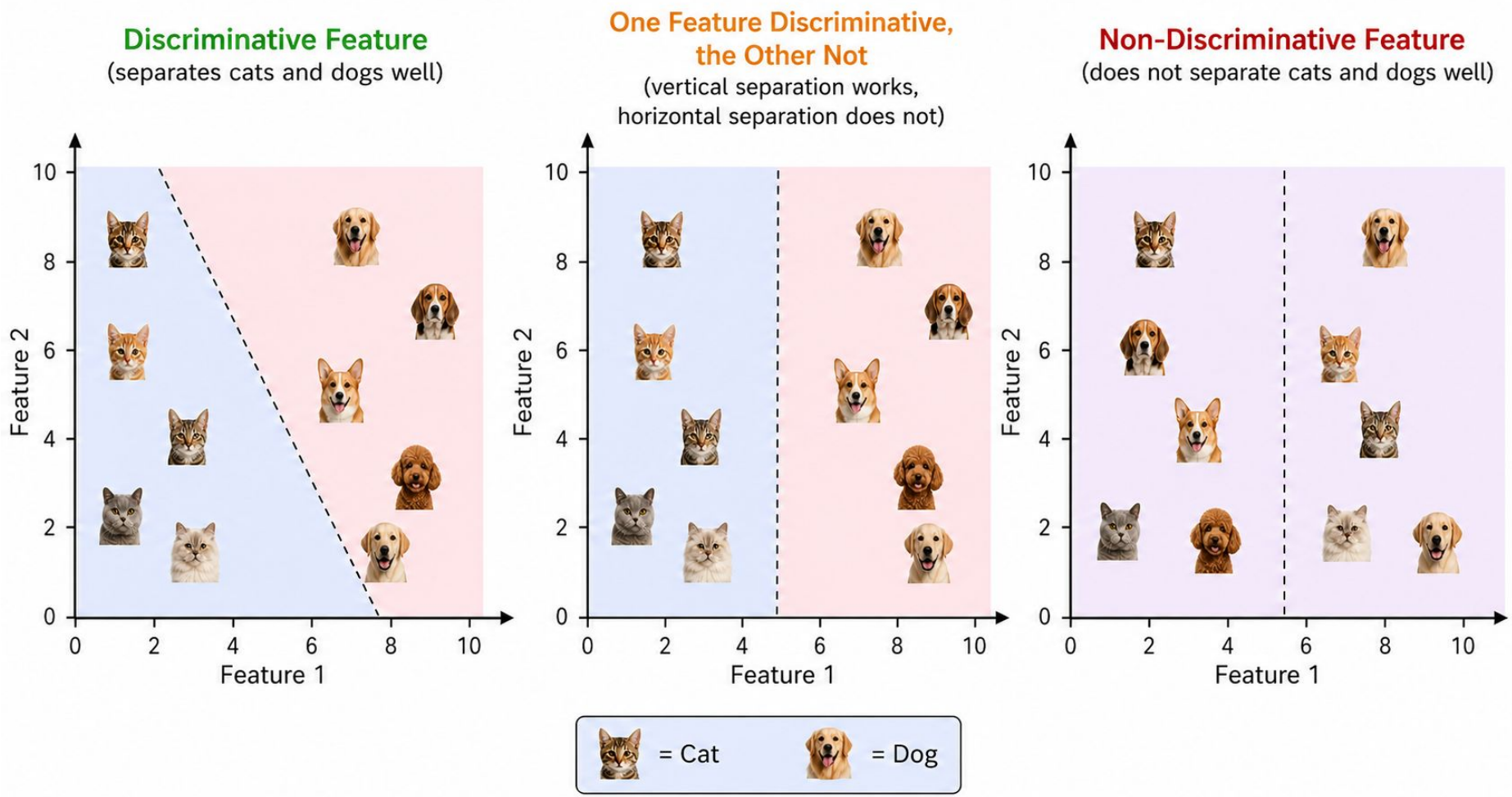
Model : depends on layout of datapoints in feature space

No “true model”, we need to explore the “best model”!

Feature space itself depends on choice of features!

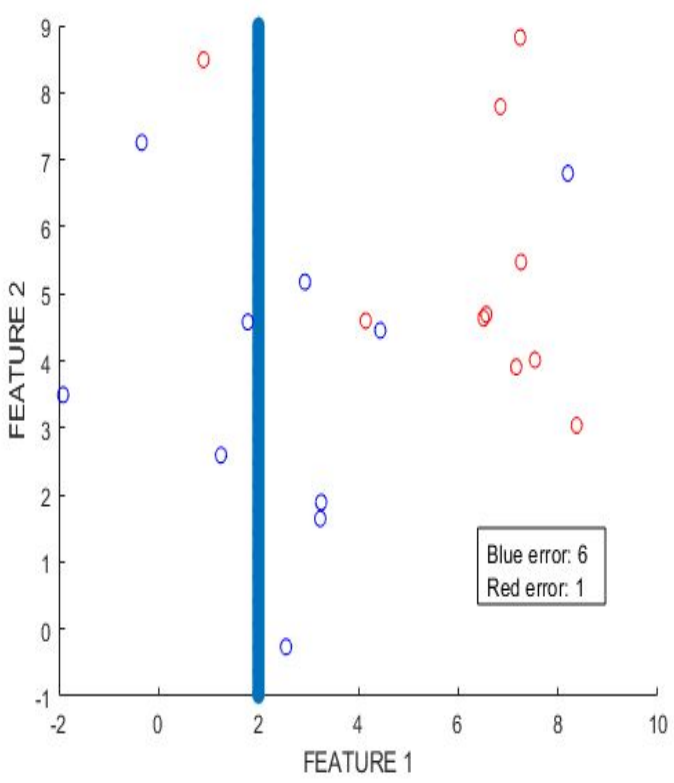
Linear Classifiers

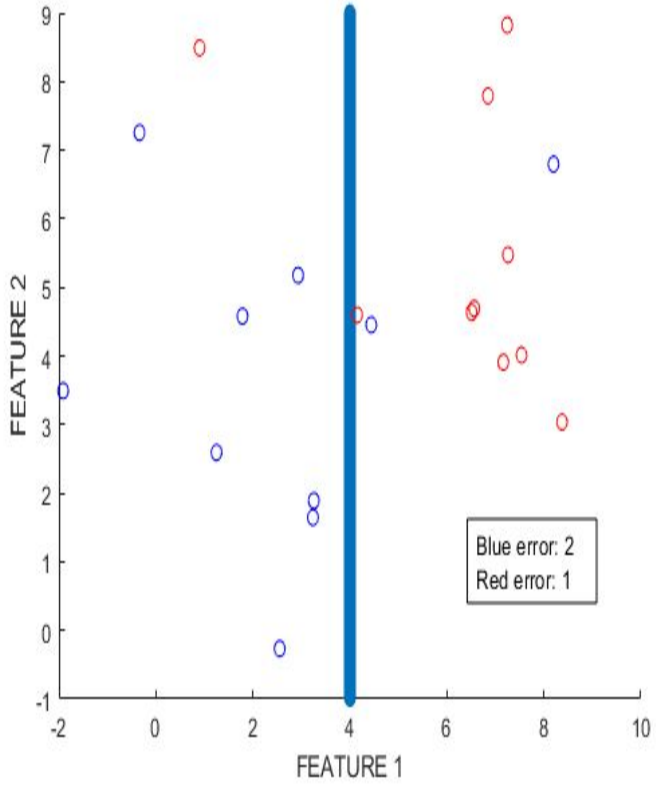
A line/hyperplane which separates the data-points from 2 classes

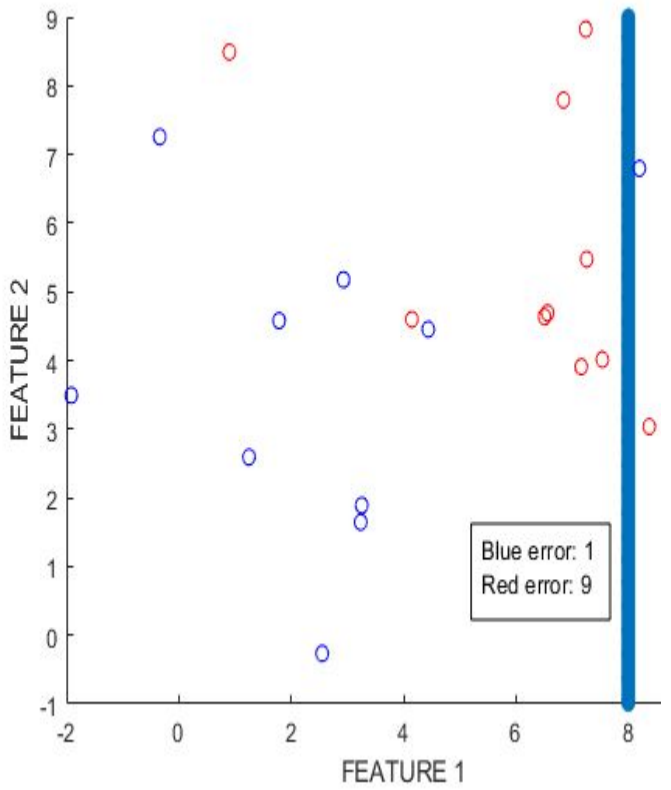


Linear Classifiers

A line/hyperplane which separates the data-points from 2 classes



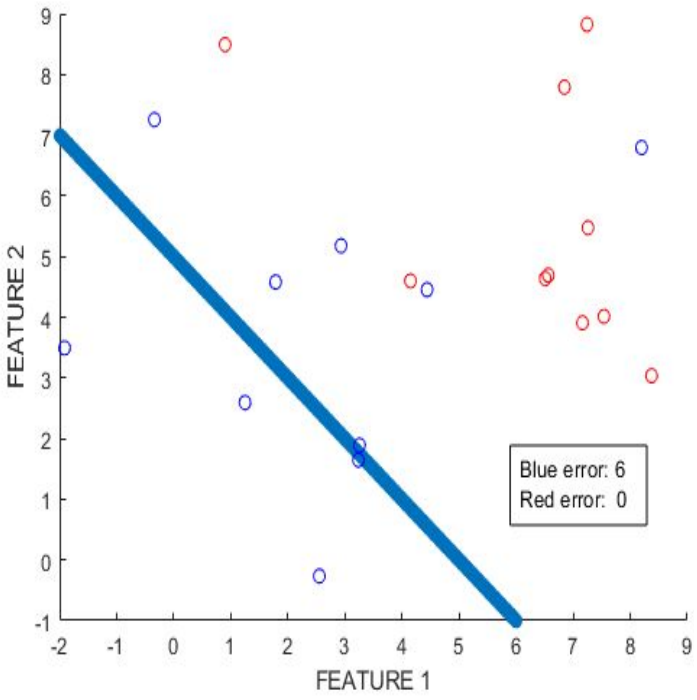


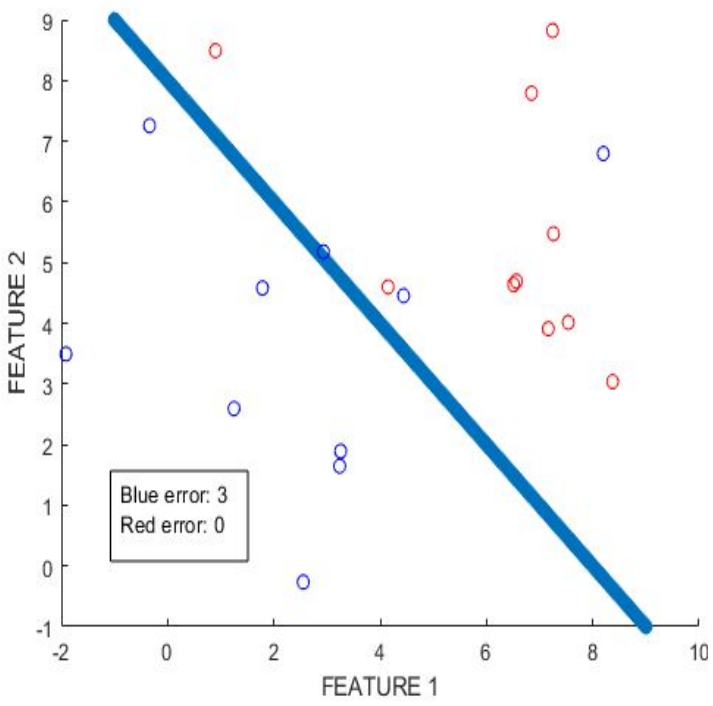


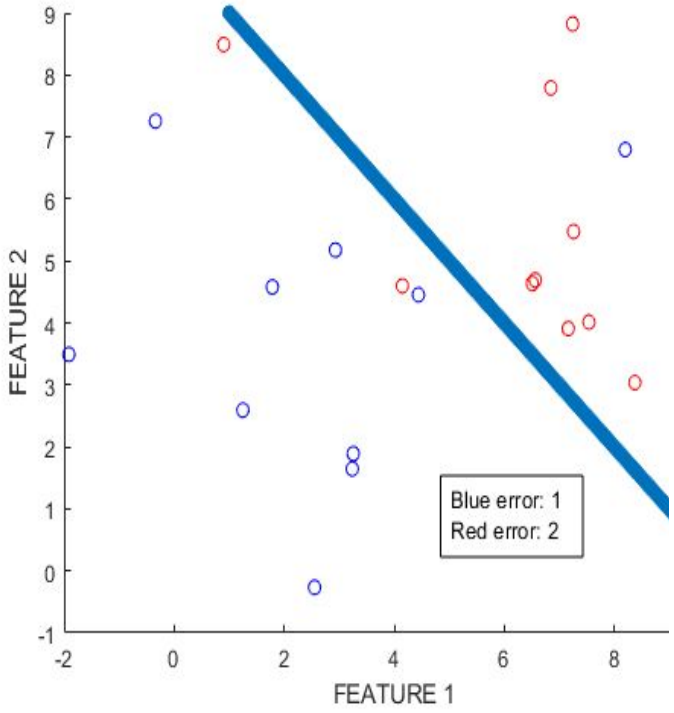
Suitable for numerical features only!

Linear Classifiers

A line/hyperplane which separates the data-points from 2 classes







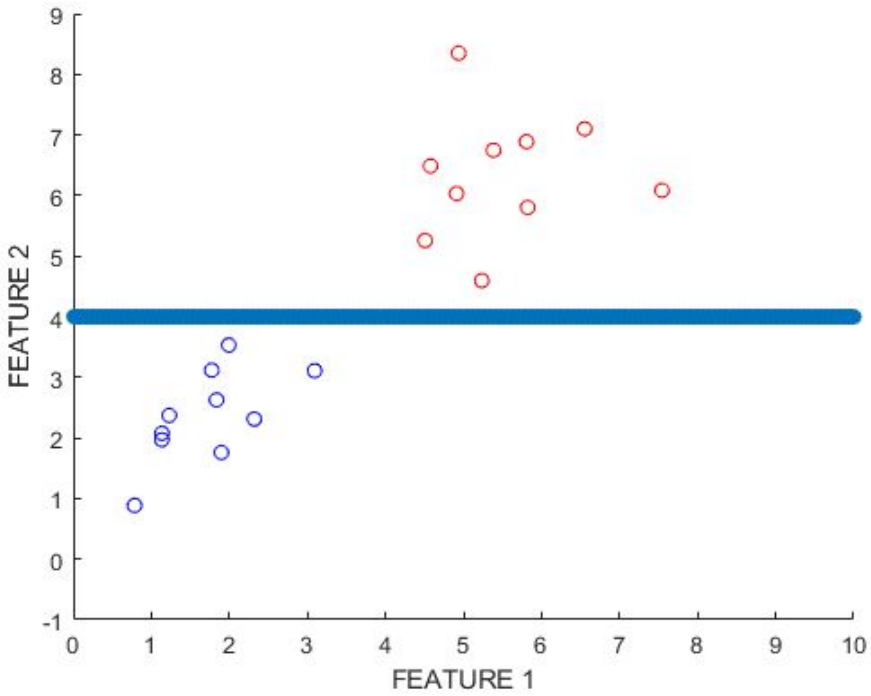
Linear Classifier Equation

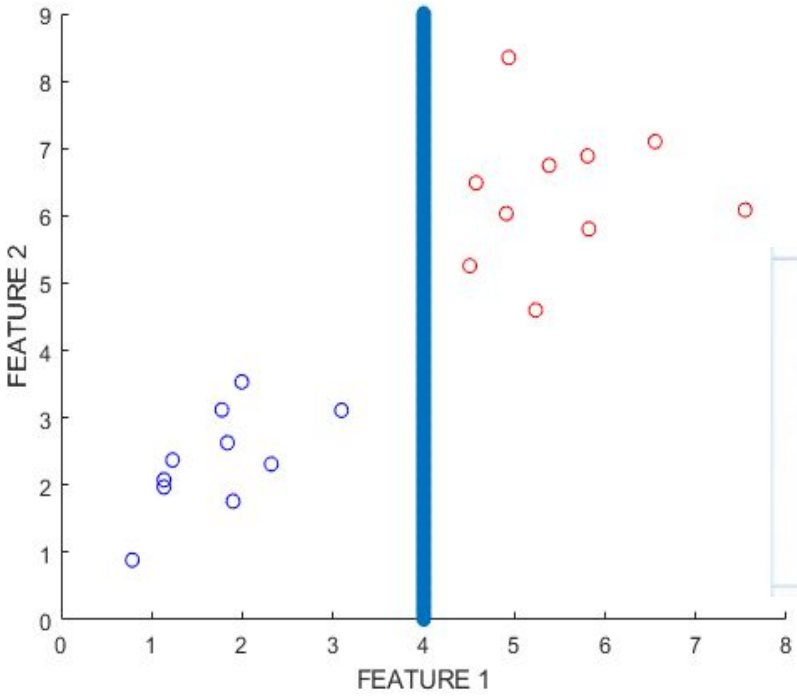
- Linear classifier in 2D plane is just a straight line
- Equation: w1x1 + w2x2 + w3 = 0 (w1,w2,w3: parameters, x1,x2 features)
- W3 is called “bias”, coefficient of “default feature” with value 1!
- Can be generalized to higher dimensions also
- Generalized form: W.X=0
- If feature space is D-dimensional, W and X are (D+1)-dimensional

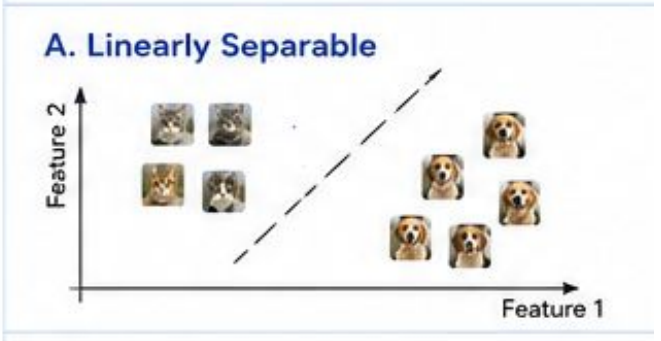
- Linear classifier: y=sign(W.X)! [+1 on one side, -1 on other!]

Linear Separability

Linearly separable: A dataset for which there exists at least one linear classifier which perfectly separates the data-points into 2 classes

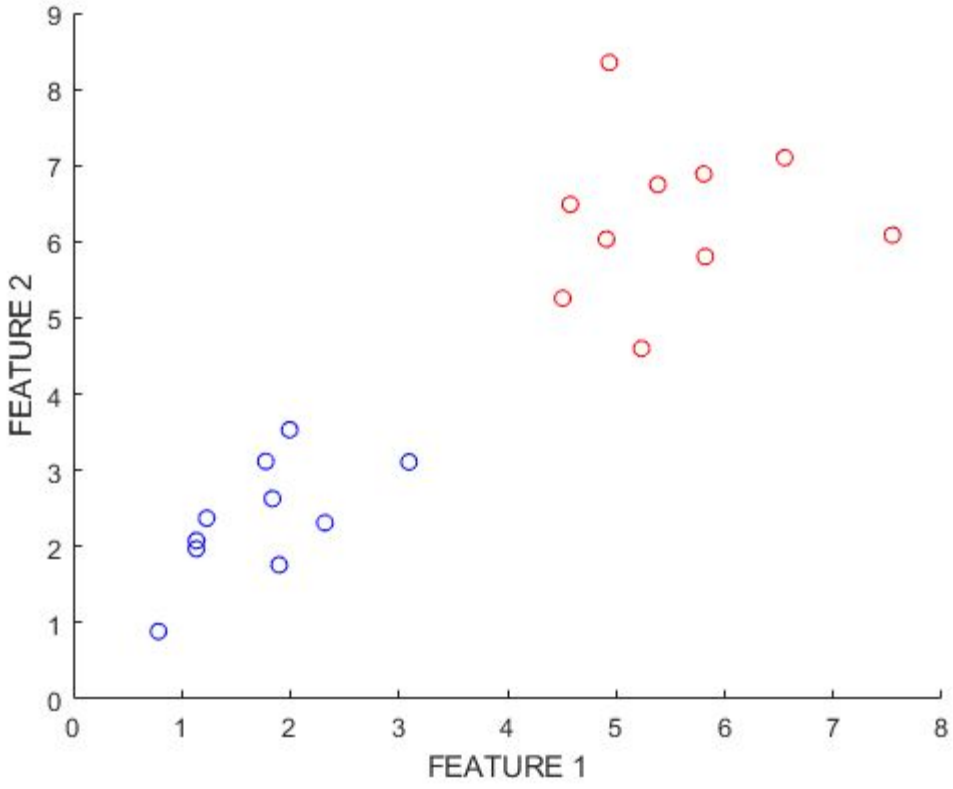




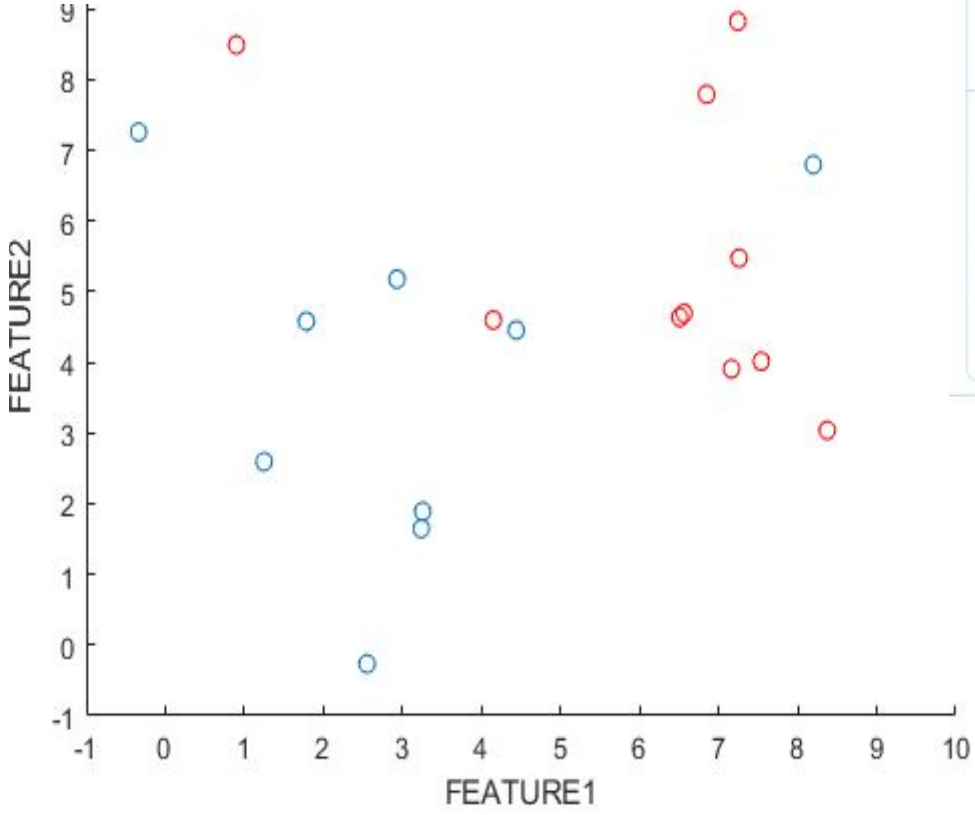


Linear Separability

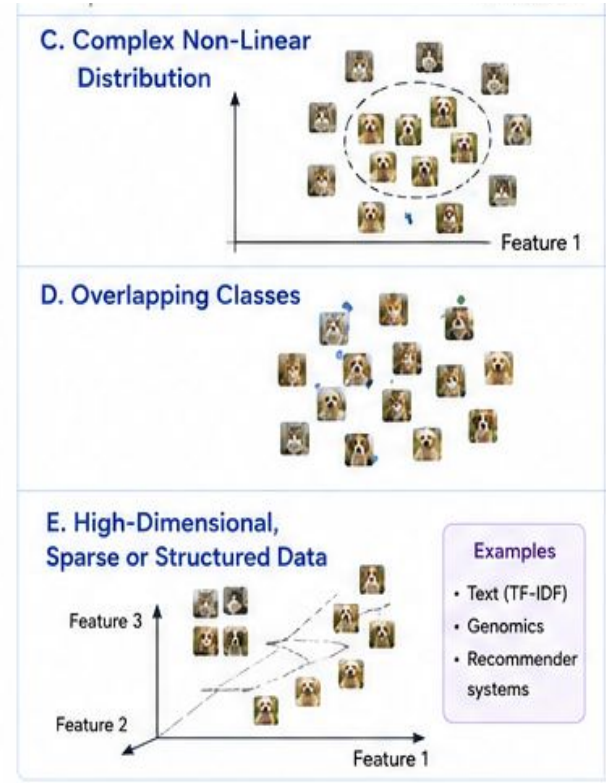
Not all datasets are linearly separable



YES



NO



Linear Classifier

- (W,b): linear classifier, Label Prediction = sign($W.X_i+b$)
- Given a data-set, there may be any number (including 0) of linear classifiers, which **perfectly** separates the two
- But how can we find them?
- Perceptron algorithm can find one of them!
- Idea: 1) Start with an estimate

  2) Take each data-point, see if it can be correctly predicted

  If not, update the estimate

  3) Stop when all data-points are correctly classified

Perceptron Algorithm

- Input: (X_1,y_1), (X_2,y_2), …… , (X_N,y_N) where y_i = +1 or -1
- Initialize W = [1 0 …. 0]
- For t = 1 to max_iter (or, till convergence)

    For i = 1 to N

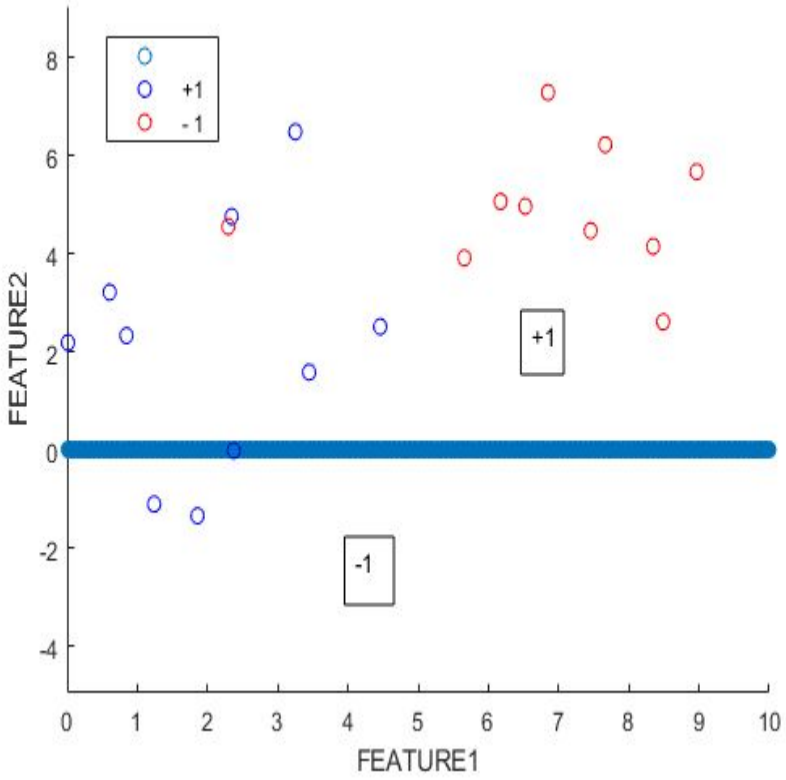
        if (y_i*sign(W.X_i)<0) ////misclassification!

            W = W + y_iX_i ///update W

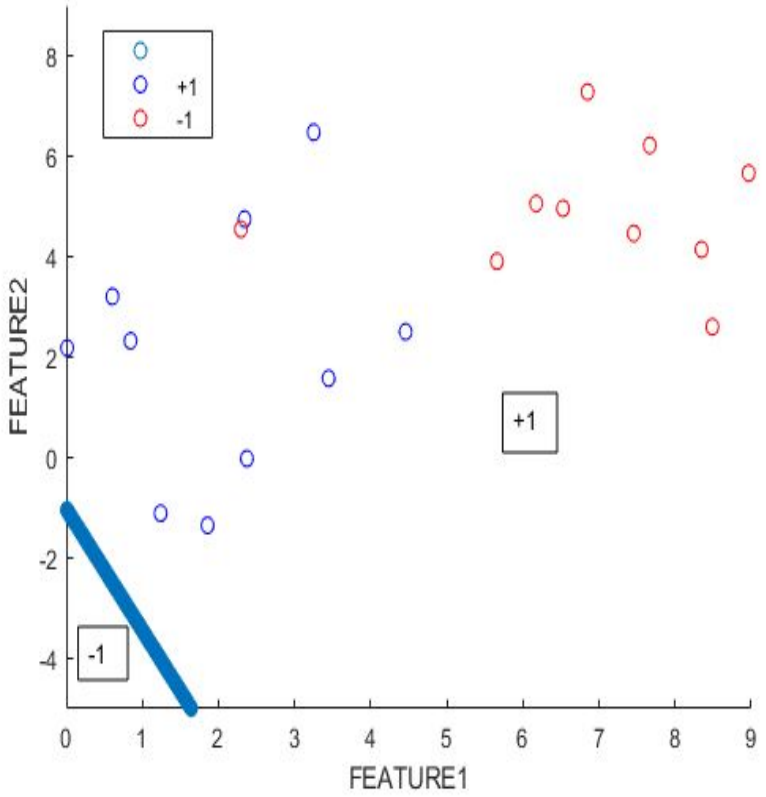
Output: final values of (W)

Finding Linear Classifier

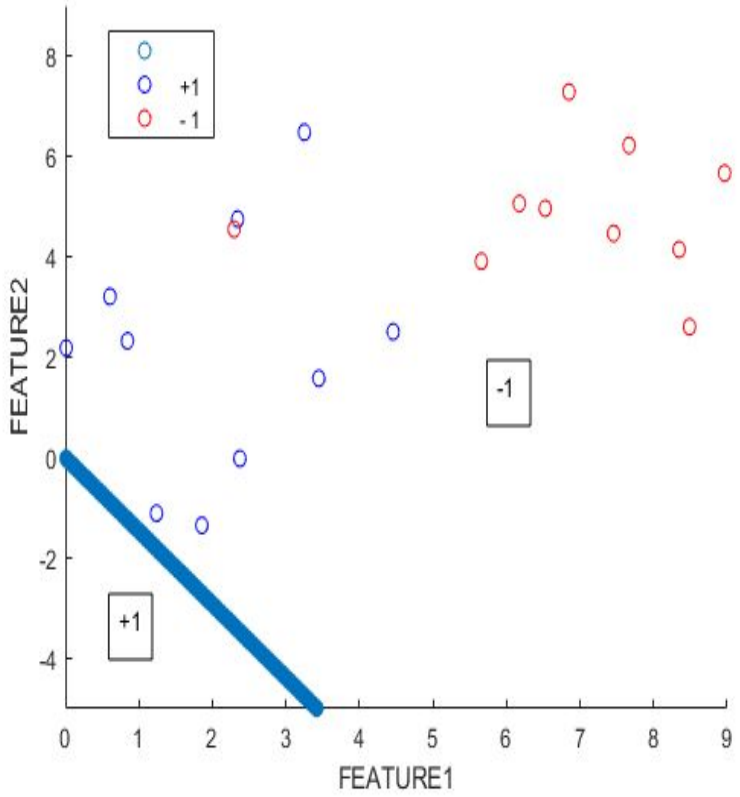
Perceptron algorithm: update temporary classifier at every misclassification



INITIALIZATION, ERROR = 13



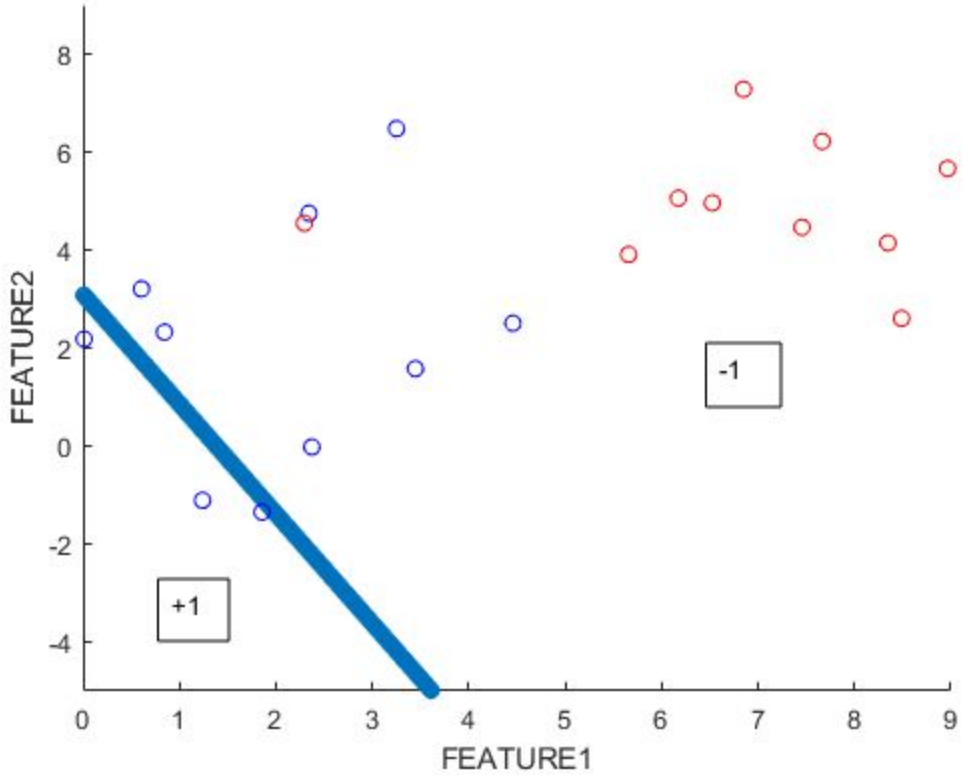
AFTER 10 POINTS, ERROR = 10



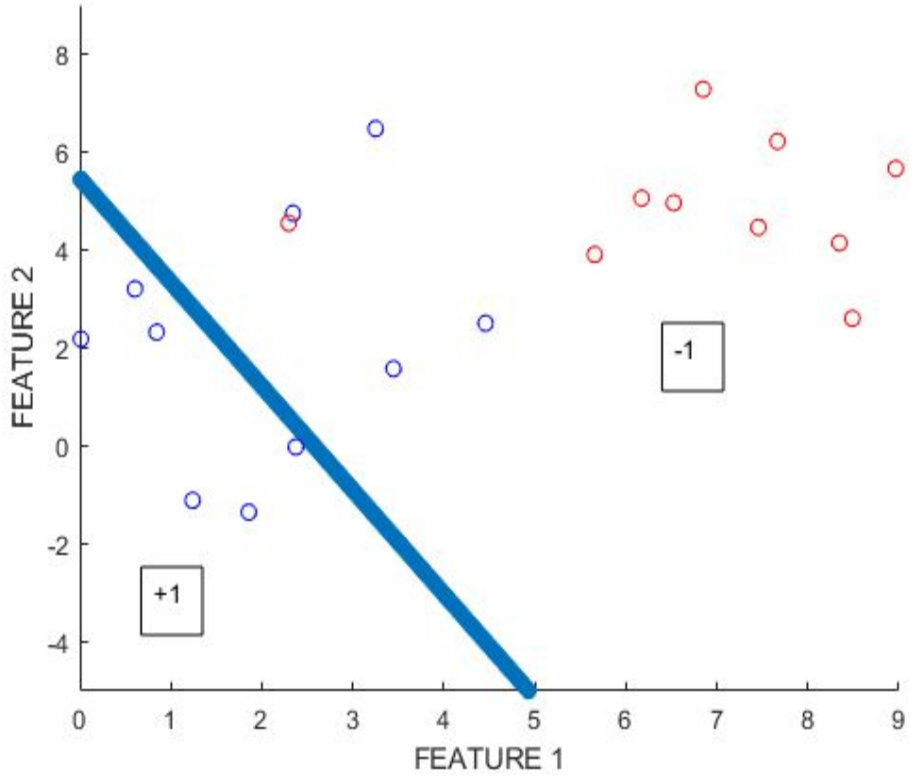
AFTER 1 ITERATION, ERROR = 10

Finding Linear Classifier

Perceptron algorithm: stop when no more/very few misclassifications!



AFTER 7 ITERATIONS, ERROR = 7



AFTER 50 ITERATIONS, ERROR = 4

If dataset is linearly separable, perceptron will find perfect linear classifier in finite iterations

Linear Classifiers are not Unique

We prefer the linear classifier to be as far away from examples as possible

A small amount of feature noise can result in misclassification

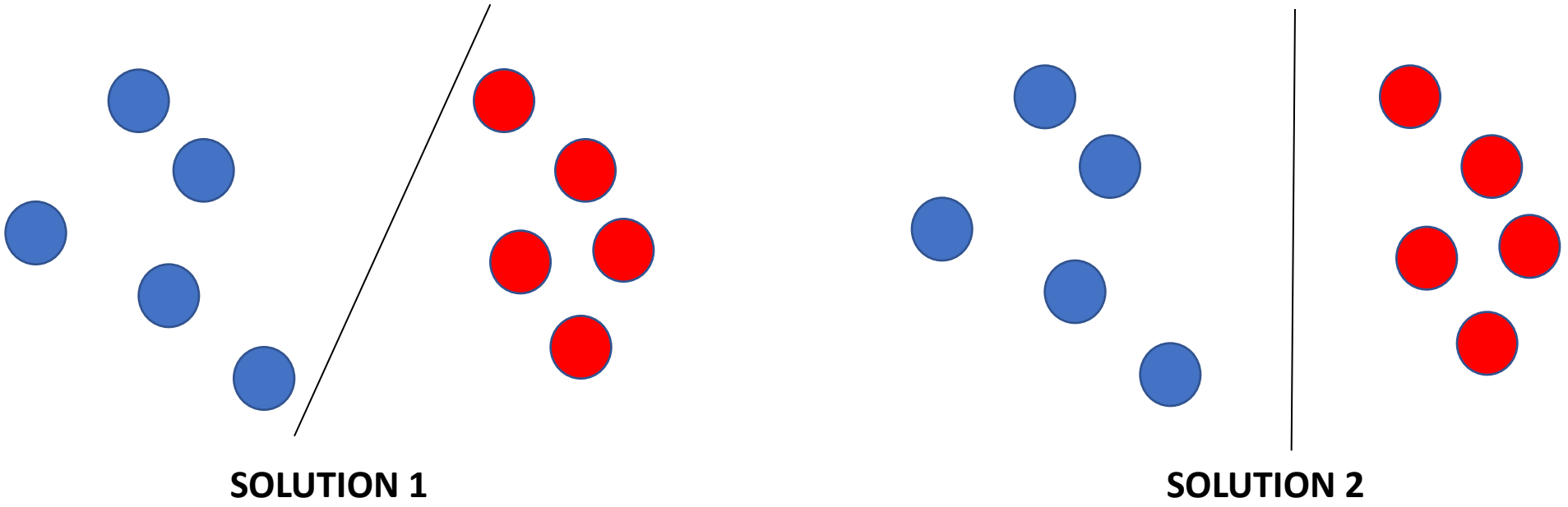


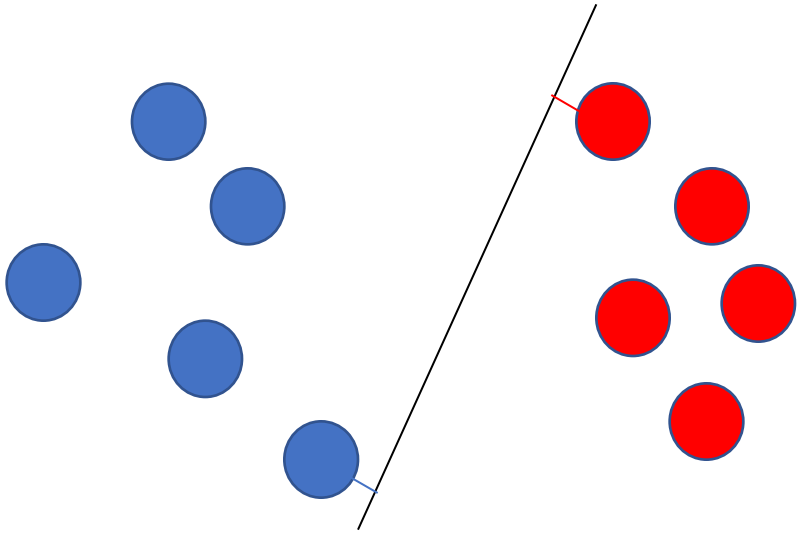
Image source: Google Images

Mentimeter 1

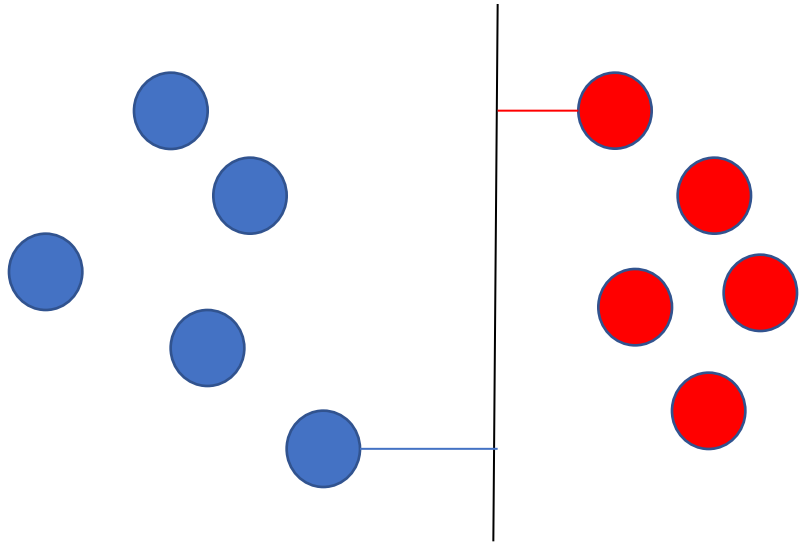
Mentimeter 2

Margin of a Linear Classifier

Margin of a class WRT a Linear Classifier W: least perpendicular distance of any example from W



SOLUTION 1
MARGIN: 1 (BLUE), 1 (RED)

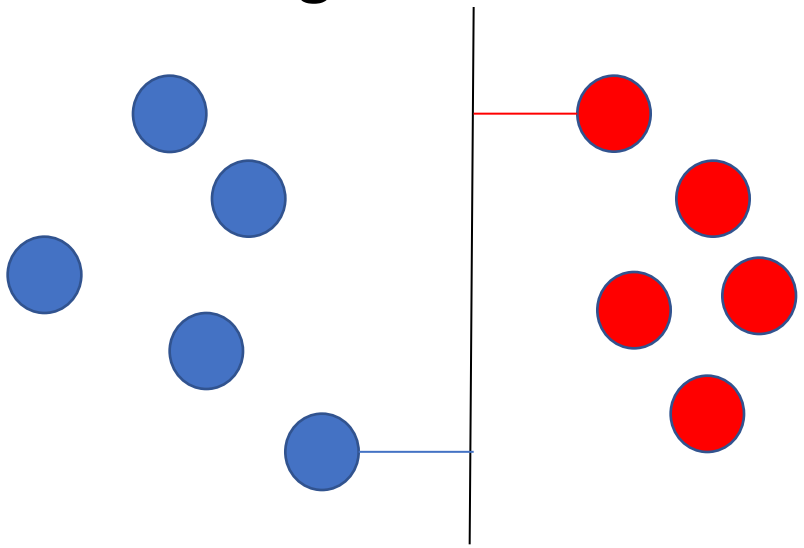


SOLUTION 2
MARGIN: 6 (BLUE), 4 (RED)

Max-Margin Linear Classifier

We wish to find a linear classifier which will be as far away as possible from both classes!

Maximize the total margin and maintain equal distance



MARGIN: 5 (BLUE), 5 (RED)
TOTAL MARGIN: 10

Image source: Google Images

Max-Margin Linear Classifier

We can imagine two parallel “boundaries” of both classes

Optimal max-margin classifier should lie exactly between them

Total Margin = $2k/||W||_2$

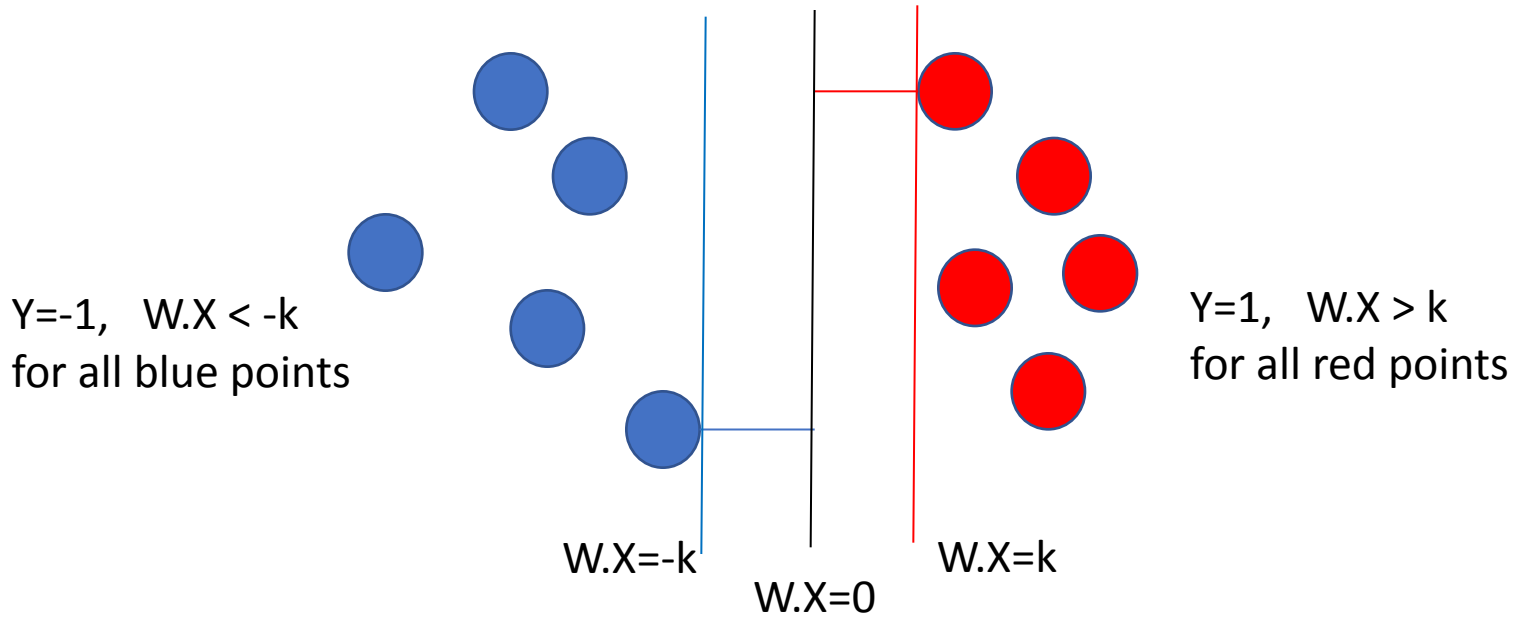


Image source: Google Images

Optimization Formulation

Aim is to find “W” for which total margin is maximized

Also all points should lie on the “correct” side of W, away from the class boundaries!

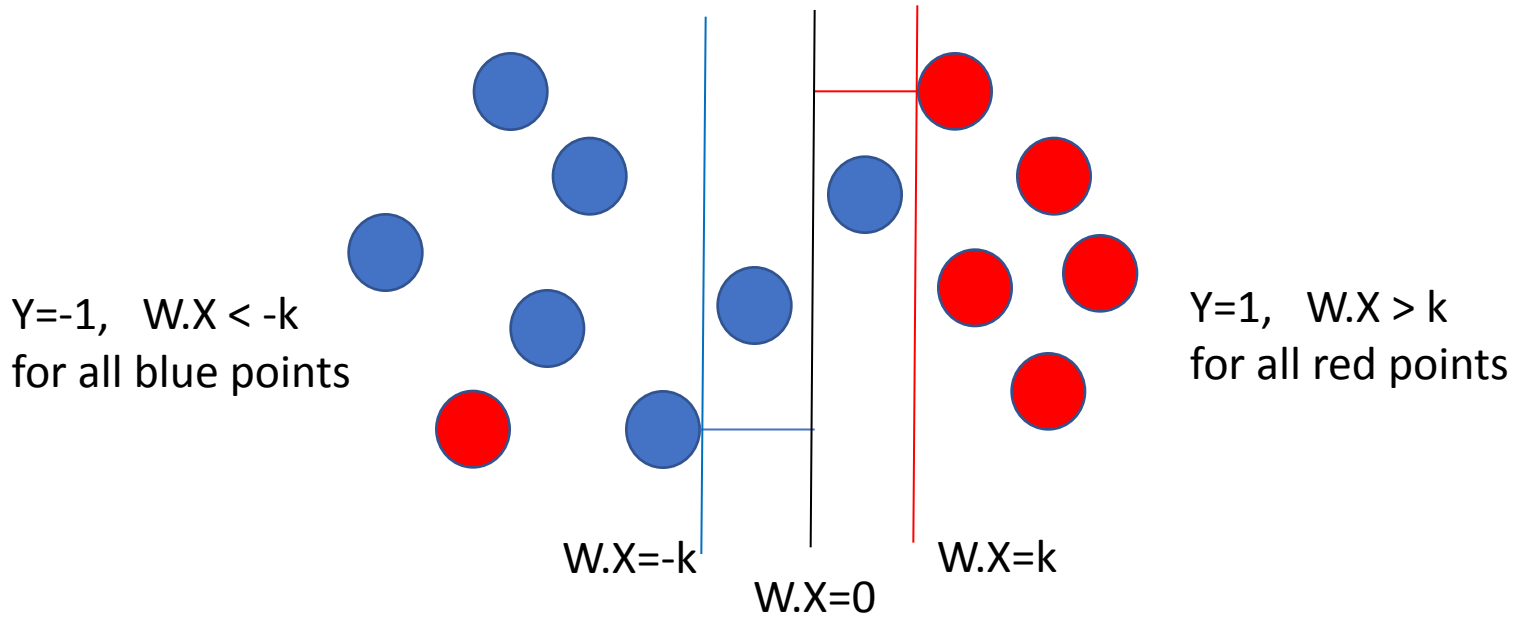
Expressed as objective function and constraints

$$W_{MM} = \text{argmin}_{W} \ \tfrac{1}{2}||W||_2^2 \ \text{ such that } y_i(W.X_i) >= K$$

Image source: Google Images

Accounting for Outliers

- Perfect classification not possible if dataset is not linearly separable
- There can be points for which constraints are violated
- Some of them can be misclassified by the optimal classifier



Soft-margin Formulation

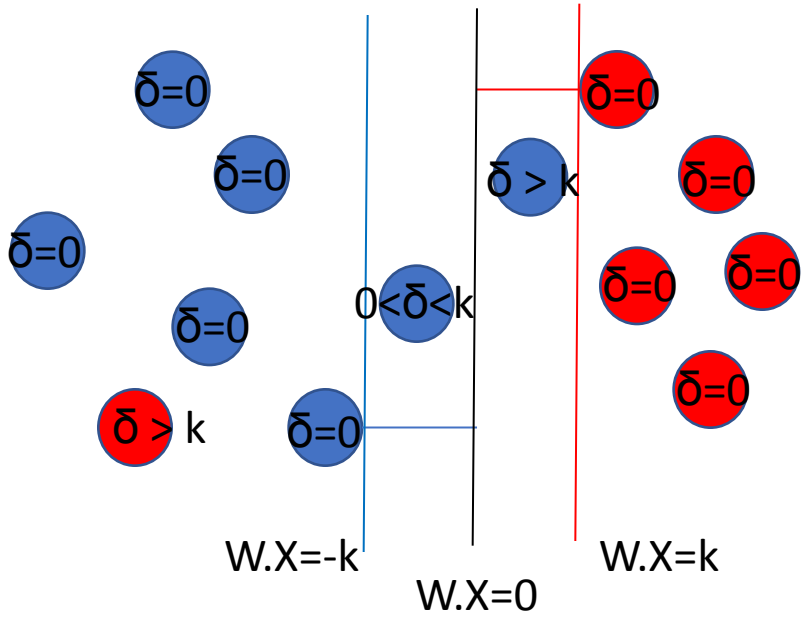
- The outliers cannot satisfy the constraint about correct classification
- They are given "grace marks" through additional "slack" variable δ

$$W_{MM} = \text{argmin}_W \ \tfrac{1}{2}||W||_2^2 + C\Sigma_i\delta_i \quad \text{such that } y_i(W.X_i) + \delta_i >= K$$

- We do not know which examples are outliers, so all examples given the slack variable δ_i
- However, we wish to minimize the outliers $\Sigma_i\delta_i$!

Soft-margin Formulation

- The outliers cannot satisfy the constraint about correct classification
- They are given “grace marks” through additional “slack” variable δ



Support Vector Machine

- These formulations can be solved by “Quadratic Programming Packages”
- Solution: $W = \Sigma_i \alpha_i Y_i X_i$
- Here α_i are additional scalar variables for each examples
- For most examples their values is 0
- They are non-zero only for those examples which are either

  i) misclassified by the optimal linear classifier

  ii) are the closest to the optimal linear classifier

The examples with non-zero α_i are called **Support Vectors**

“W” depends on them only

Classification by SVM

- Given new feature vector Xtest, we need to predict its label
- $Y = \text{sign}(W^T X_{test})$ where $W = \Sigma_i \alpha_i Y_i X_i$
- So, $Y = \text{sign}(\Sigma_i \alpha_i Y_i X_i{}^T X_{test})$
- Basically, we need to calculate the dot-product of test feature vector with those of all support vectors!

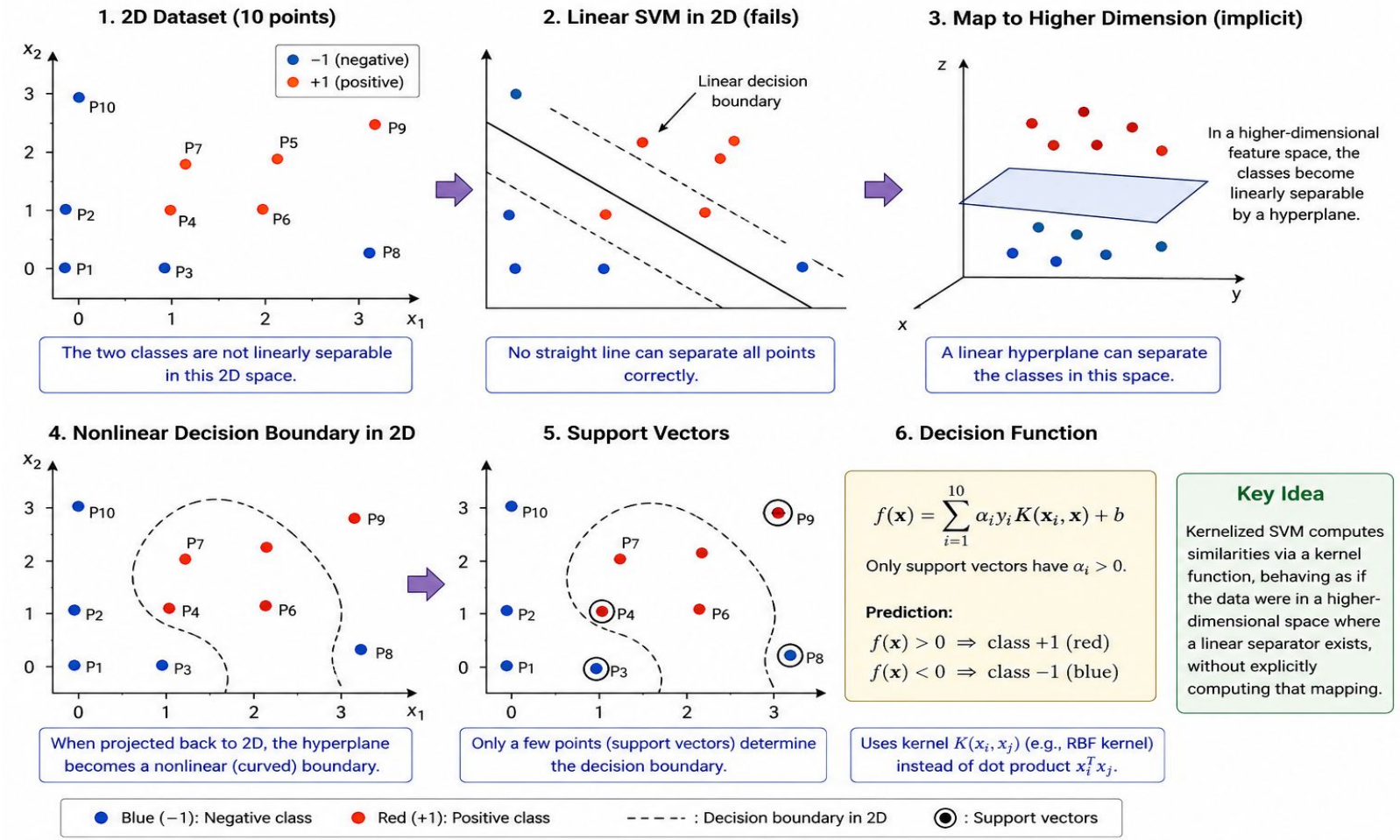
Non-linear SVM Classification

- Examples of both classes may be separable by a non-linear boundary
- They may become linearly separable after non-linear transformation of features

$$Y = \text{sign}(\Sigma_i \alpha_i Y_i \varphi(X_i)^T \varphi(X_{test}))$$
$$= \text{sign}(\Sigma_i \alpha_i Y_i K(X_i, X_{test}))$$

Where K is called Kernel Function

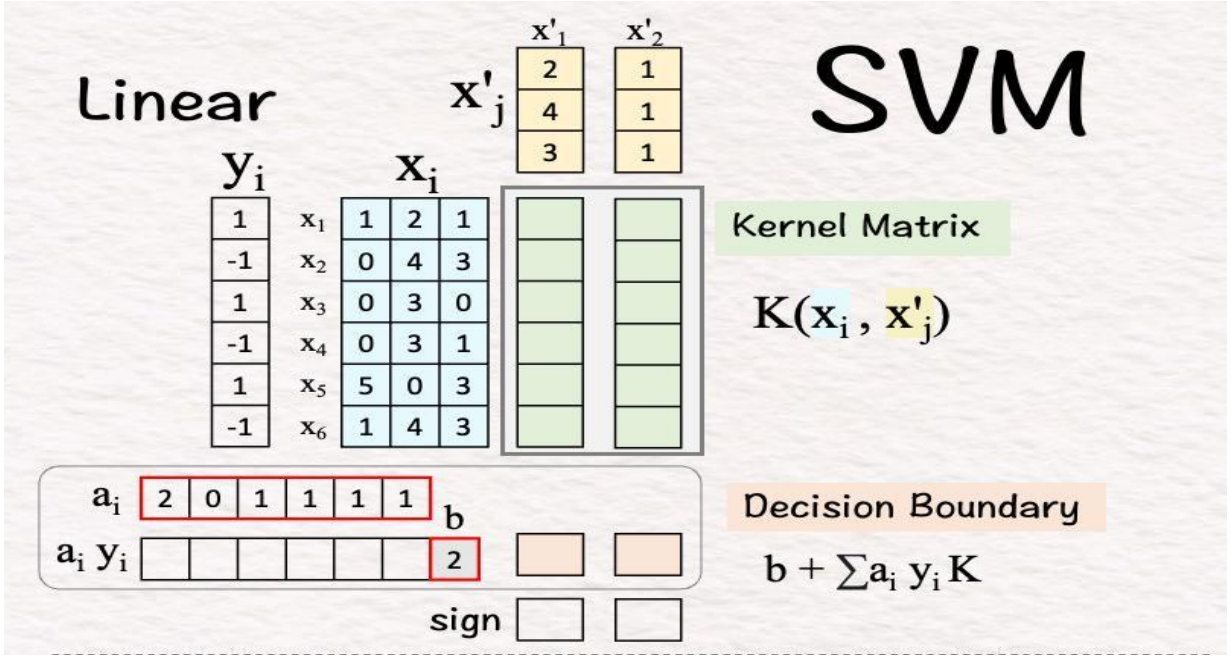
“Kernelized” SVM!



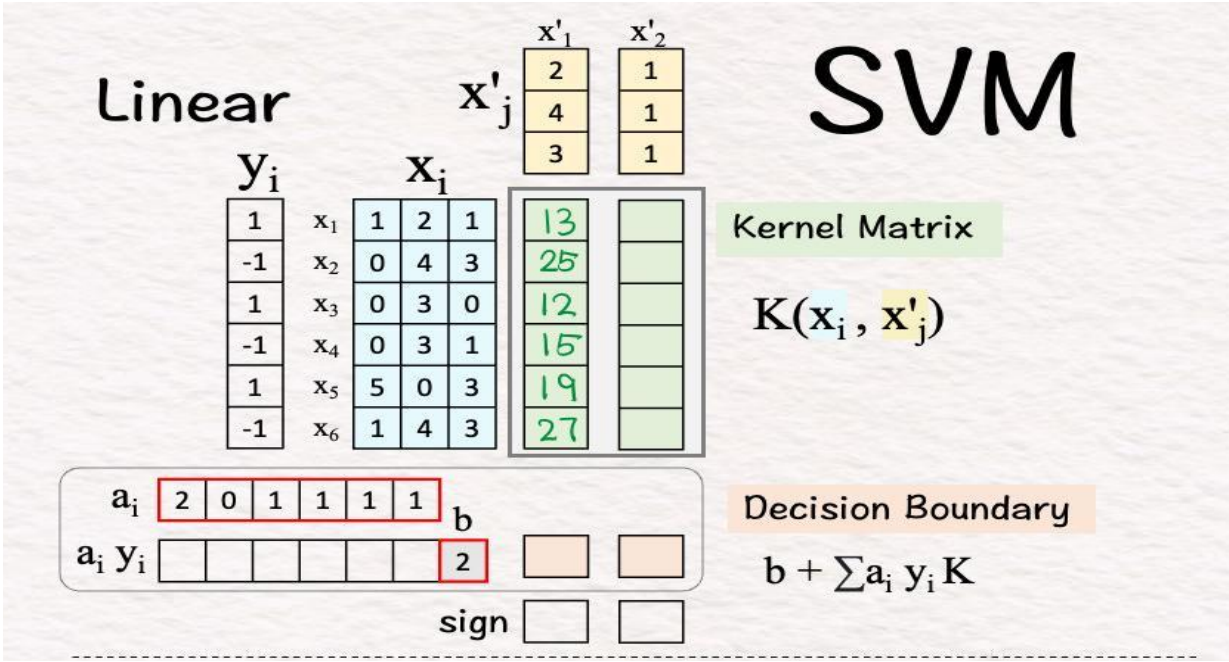
Non-linear SVM Classification

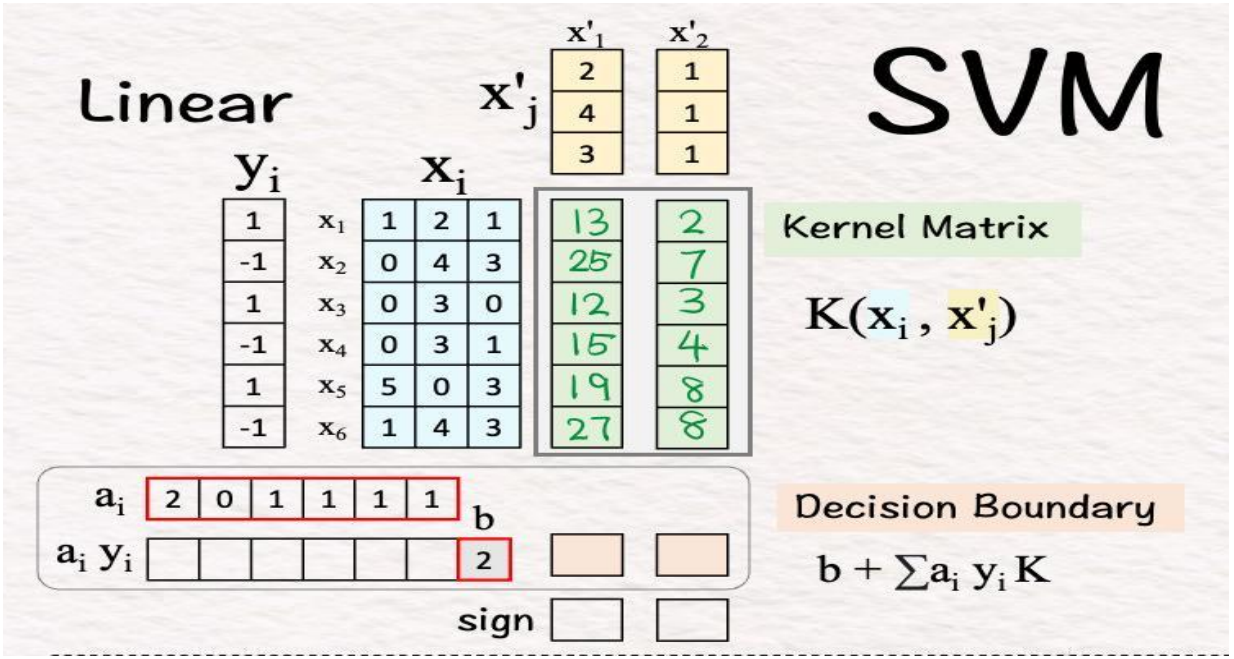
Kernel	Formula	Intuition
1. Linear Kernel (baseline) Produces a straight-line boundary.	$K(\boldsymbol{x}, \boldsymbol{z}) = \boldsymbol{x}^\top \boldsymbol{z}$	Equivalent to the standard inner product. SVM finds the best separating straight line in 2D.
2. Polynomial Kernel Allows curved (nonlinear) decision boundaries.	$K(\boldsymbol{x}, \boldsymbol{z}) = (\gamma\, \boldsymbol{x}^\top \boldsymbol{z} + r)^d$ (common choice: $\gamma > 0,\ r \geq 0,\ d \in \mathbb{N}$)	Maps data to a polynomial feature space where classes may become linearly separable.
3. RBF (Gaussian) Kernel Very flexible; can model complex, smooth boundaries.	$K(\boldsymbol{x}, \boldsymbol{z}) = \exp\left(-\gamma \left\|\boldsymbol{x} - \boldsymbol{z}\right\|^2\right)$ $(\gamma > 0)$	Similarity depends on distance. Points close to each other have high similarity. Produces localized influence and smooth nonlinear boundaries.
4. Sigmoid Kernel Behaves like a neural network activation; boundary can be nonlinear.	$K(\boldsymbol{x}, \boldsymbol{z}) = \tanh(\gamma\, \boldsymbol{x}^\top \boldsymbol{z} + r)$ $(\gamma > 0)$	Equivalent to a two-layer neural network (without hidden layer training). Can produce complex boundaries.

- Different kernel functions may be suitable for different datasets
- We can define our own Kernels based on applications
- **Multiple Kernel Learning**: use a combination of multiple kernels to define the decision function!

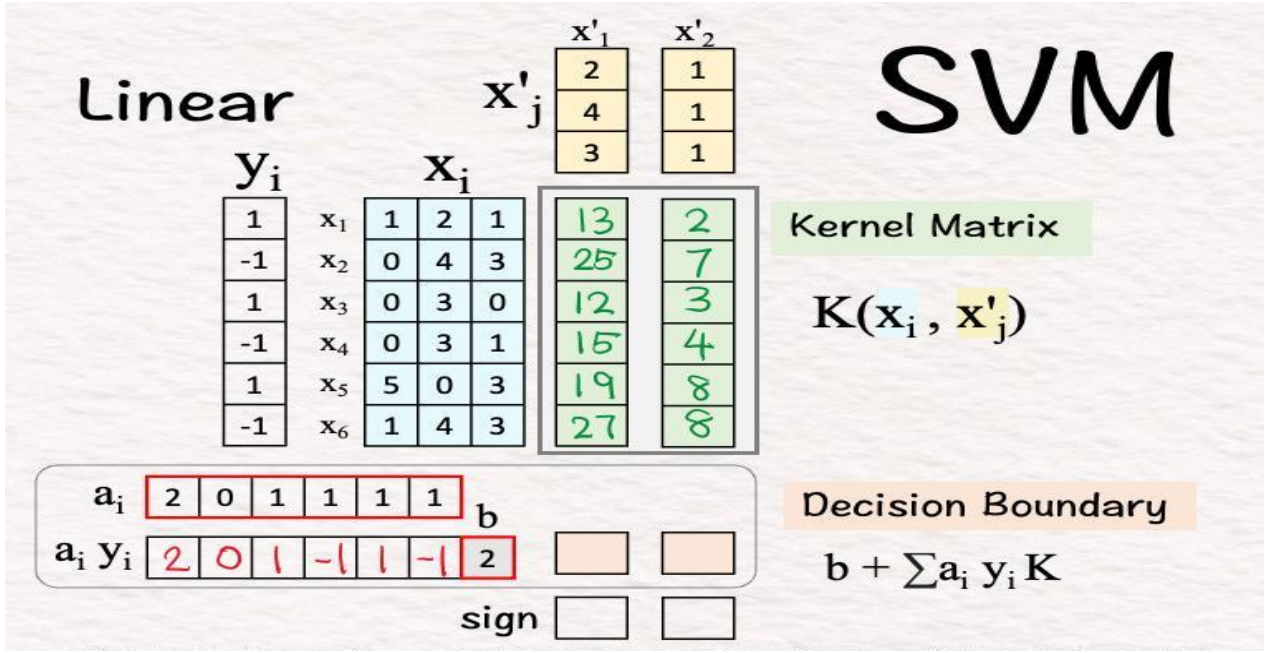


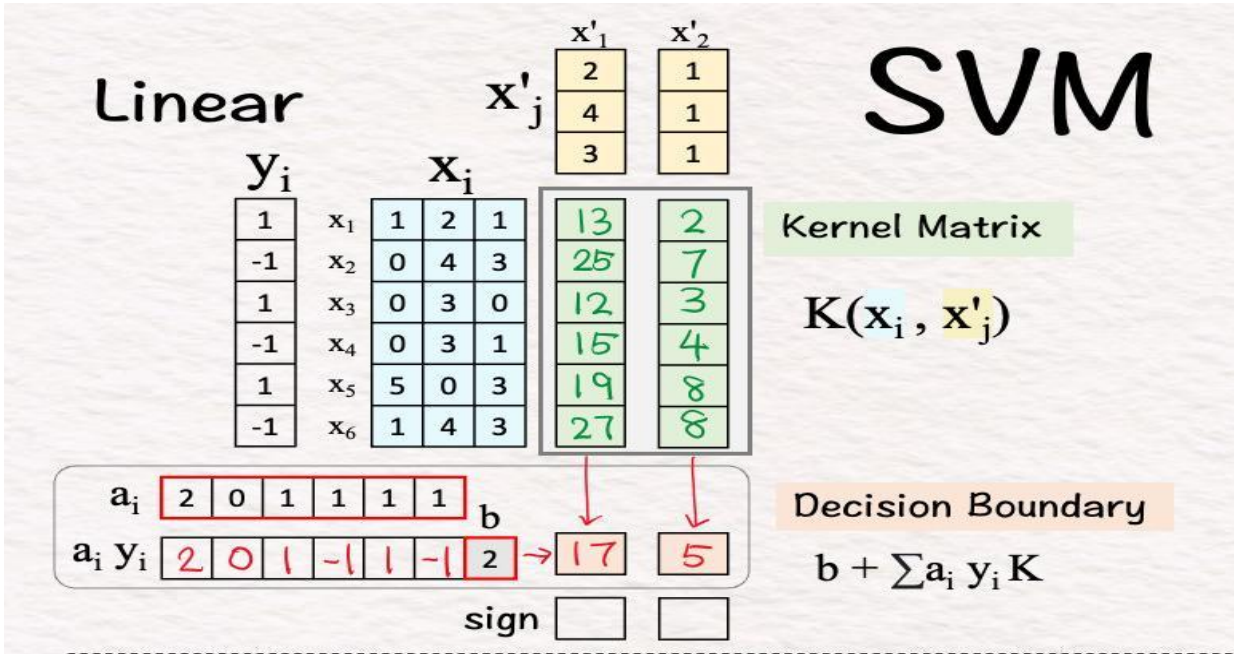
Credits: Prof. Tom Yeh
AI by Hand



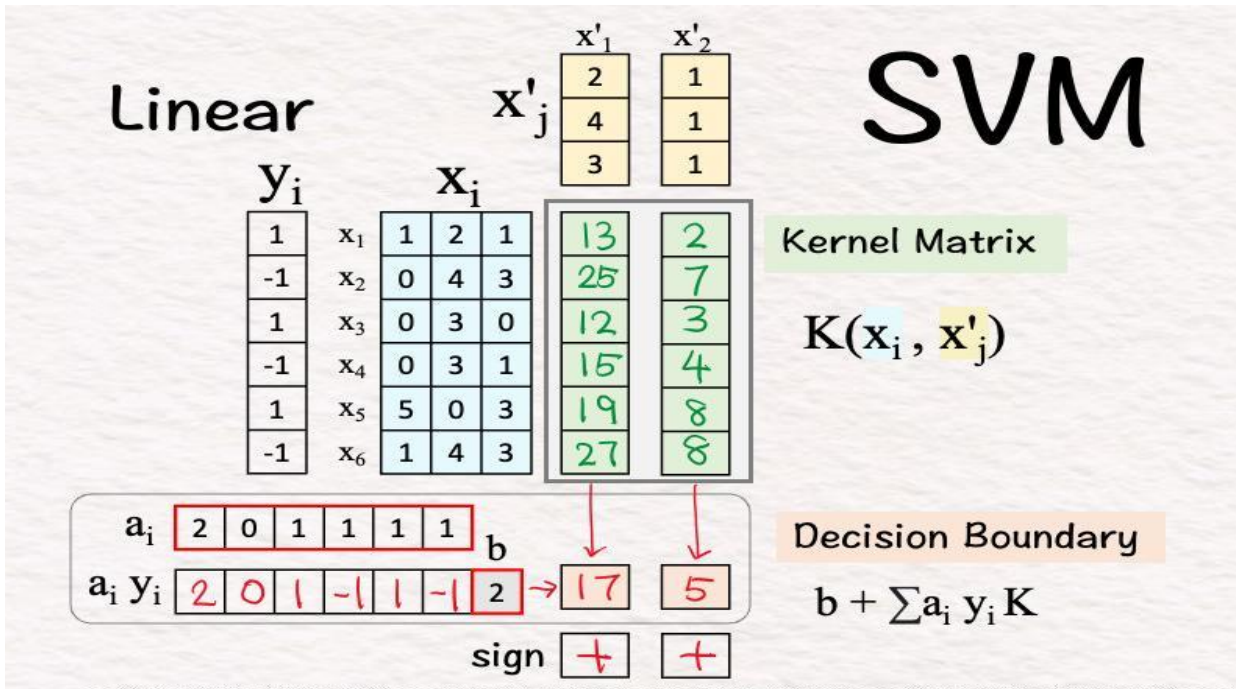


Credits: Prof. Tom Yeh
AI by Hand





Credits: Prof. Tom Yeh
AI by Hand



Mentimeter 3

Mentimeter 4

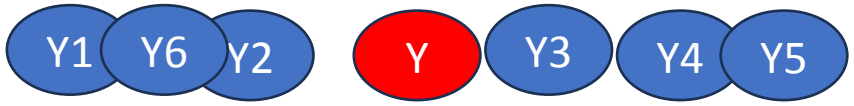
Bias of Models

Training Datasets	D1	D2	D3	D4	D5	D6
Learned Models	f1	f2	f3	f4	f5	f6
Predictions	f1(X)	f2(X)	f3(X)	f4(X)	f5(X)	f6(X)
Errors	E1=Y-f1(X)	E2=Y-f2(X)	E3=Y-f3(X)	E4=Y-f4(X)	E5=Y-f5(X)	E6=Y-f6(X)

Ensemble Mean Prediction g(X) = (1/M)*(f1(X) + f2(X) + ……. + fM(X))

Bias (X) = Y – g(X) [Mean of errors by ensemble members]

Bias (X) = 0: ensemble mean prediction is the true label!



Ensemble mean close to true value: LOW BIAS



Ensemble mean away to true value: HIGH BIAS

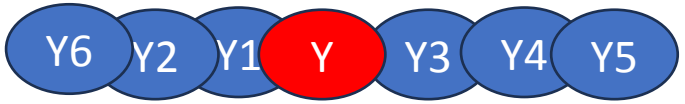
Bias of Model Family= (1/N)*(Bias(X1) + Bias(X2) + …….. Bias(XN))

Variance of Models

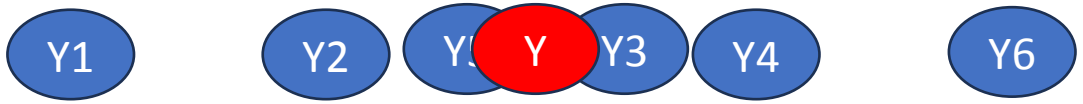
Training Datasets	D1	D2	D3	D4	D5	D6
Learned Models	f1	f2	f3	f4	f5	f6
Predictions	f1(X)	f2(X)	f3(X)	f4(X)	f5(X)	f6(X)
Mean Deviation	f1(X) – g(X)	f2(X) – g(X)	f3(X) – g(X)	f4(X) – g(X)	f5(X) – g(X)	f6(X) – g(X)

Ensemble Mean Prediction $g(X) = (1/M)*(f1(X) + f2(X) + \ldots\ldots + fM(X))$

Variance $(X) = (1/M)*(Y - g(X))^2$



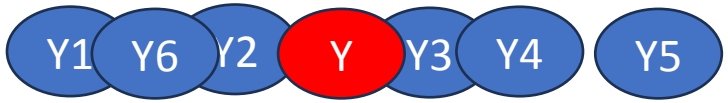
Ensemble predictions close to Ensemble Mean: LOW Variance



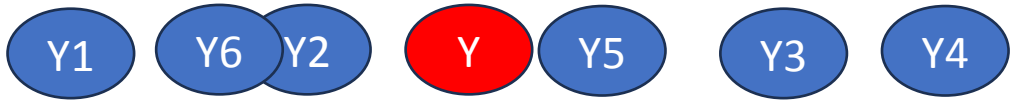
Ensemble predictions spread out from Ensemble Mean: HIGH Variance

Variance of Model Family= (1/N)*(Variance(X1) + Variance(X2) + …….. Variance(XN))

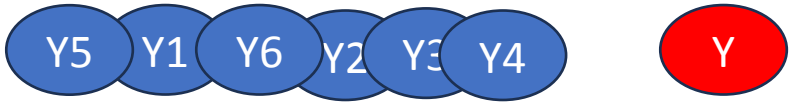
Bias vs Variance



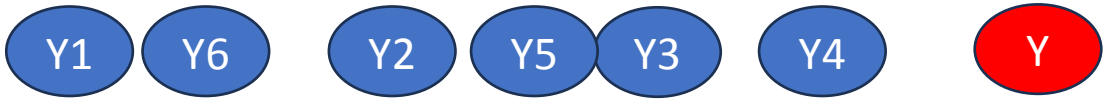
LOW BIAS, LOW VARIANCE (IDEAL)



LOW BIAS, HIGH VARIANCE



HIGH BIAS, LOW VARIANCE



HIGH BIAS, HIGH VARIANCE

- High Bias: model predictions differ from true value, irrespective of training set (Inference: Model not suitable for data)
- High Variance: model predictions differ from each other for different training sets (Inference: Model overfits on training set)

Bias-variance Tradeoff (Regression)

- Numerical example: X=[3, 5], Y= 12
- Model: Y = W0 + W1*X1 + W2*X2 [linear regression]

Dataset	W0	W1	W2	f(X)	Error Y- f(X)	Mean Deviation f(X)- g(X)	Squared Mean Deviation
D1	0.5	2.1	0.9	11.3	0.7	-0.85	0.7225
D2	0.9	1.8	1.0	11.3	0.7	-0.85	0.7225
D3	1.2	2.0	1.5	14.7	-2.7	2.55	6.5025
D4	0.8	2.5	0.6	11.3	0.7	-0.85	0.7225
Mean				12.15	0.15 (BIAS)	0	2.17 (VARIANCE)

- LOW BIAS, LOW VARIANCE!
- Can consider mean bias and variance over many examples!

Bias-variance Tradeoff (Regression)

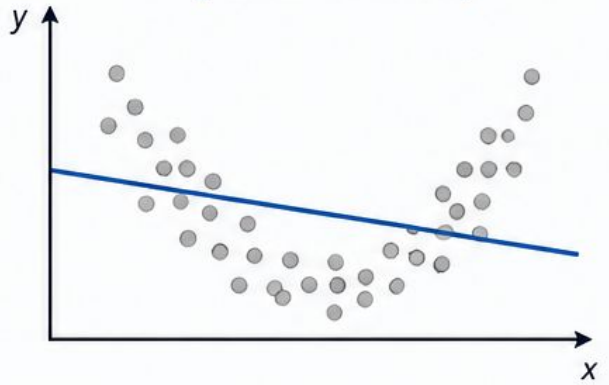
- Numerical example: X=[3, 5], Y= 18
- Model: Y = W0 + W1*X1 + W2*X2 [linear regression]

Dataset	W0	W1	W2	f(X)	Error Y- f(X)	Mean Deviation f(X)- g(X)	Squared Mean Deviation
D1	0.5	2.1	0.9	11.3	6.7	-0.85	0.7225
D2	0.9	1.8	1.0	11.3	6.7	-0.85	0.7225
D3	1.2	2.0	1.5	14.7	3.3	2.55	6.5025
D4	0.8	2.5	0.6	11.3	6.7	-0.85	0.7225
Mean				12.15	1.35 (BIAS)	0	2.17 (VARIANCE)

- HIGH BIAS, LOW VARIANCE! (Wrong choice of model?)
- Can consider mean bias and variance over many examples!

Bias-Variance vs Overfitting-Underfitting

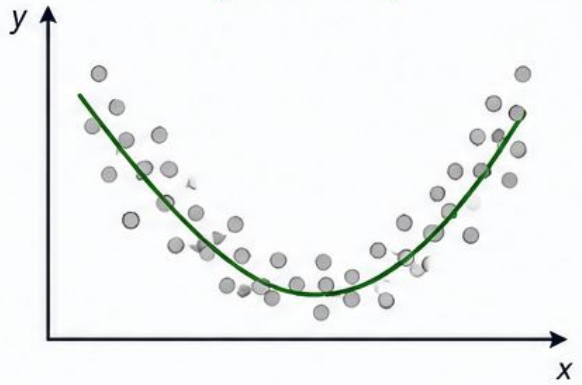
HIGH BIAS (UNDERFITTING)



- Model is too simple to capture the underlying pattern
- High training error
- High test error

Bias: High
Variance: Low

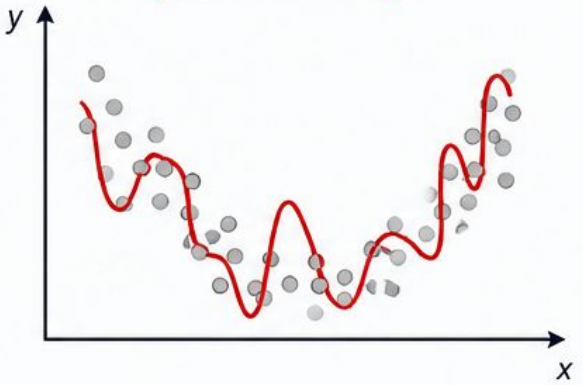
LOW BIAS, LOW VARIANCE (GOOD FIT)



- Model captures the underlying pattern
- Low training error
- Low test error

Bias: Low
Variance: Low

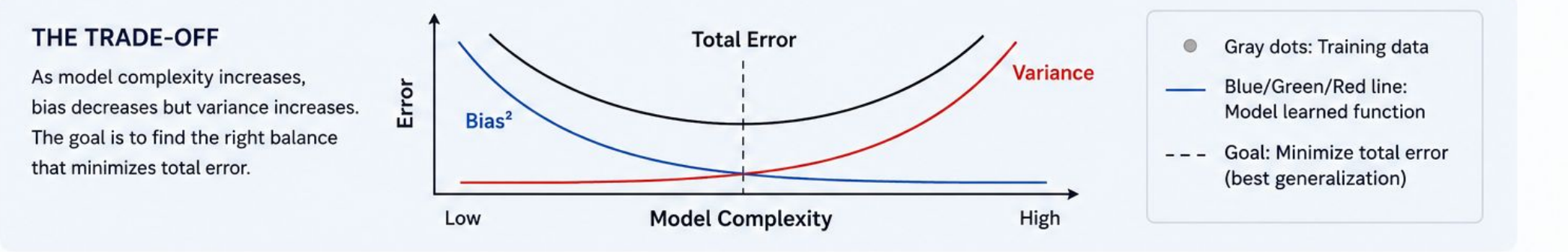
HIGH VARIANCE (OVERFITTING)



- Model is too complex and fits noise
- Low training error
- High test error

Bias: Low
Variance: High

Bias-Variance vs Overfitting-Underfitting



$$\text{Error}(X) = (Y - f(X))^2 = \text{Bias}(X)^2 + \text{Variance}(X)$$

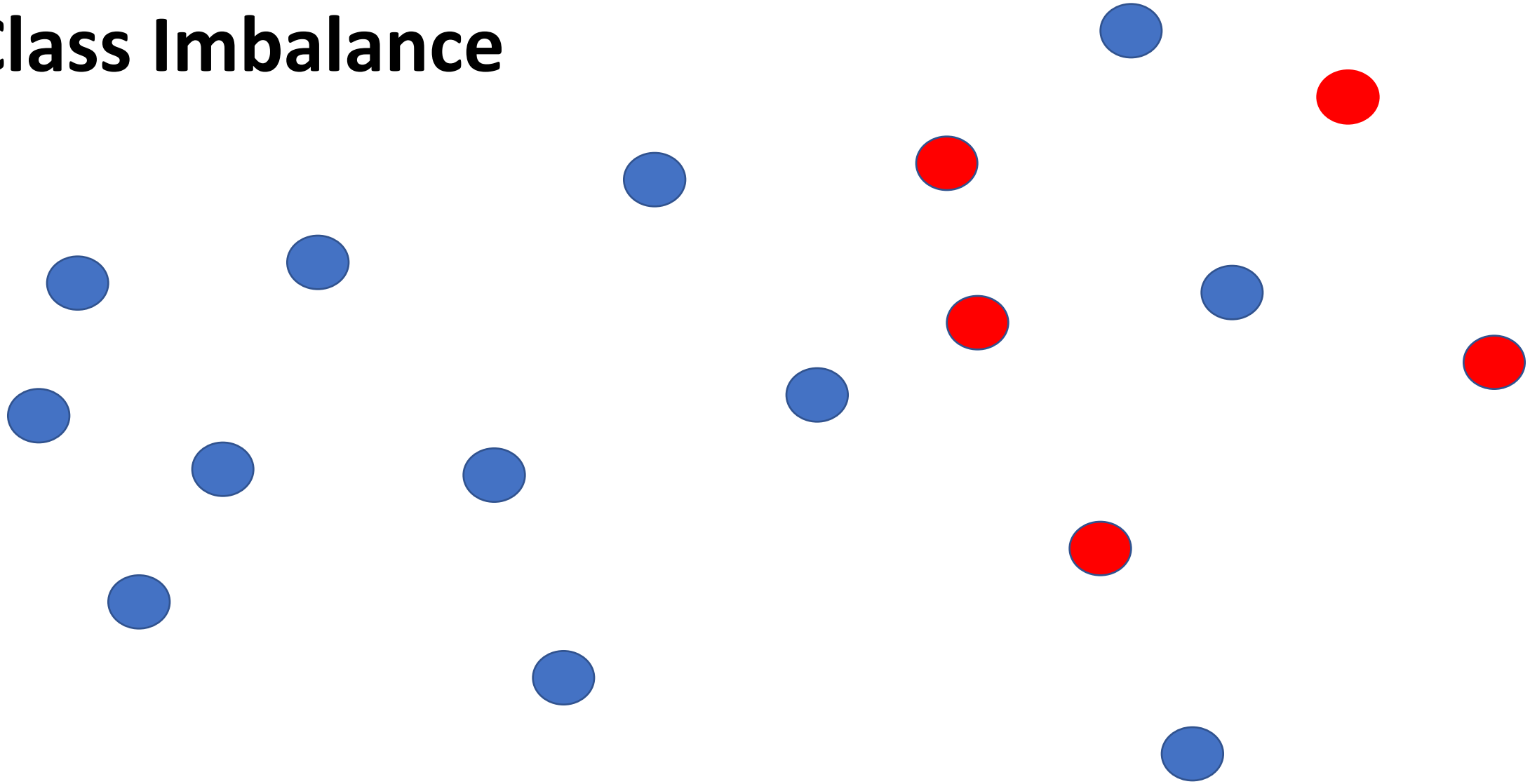
High error due to high bias: model underfitting!

High error due to high variance: model overfitting!

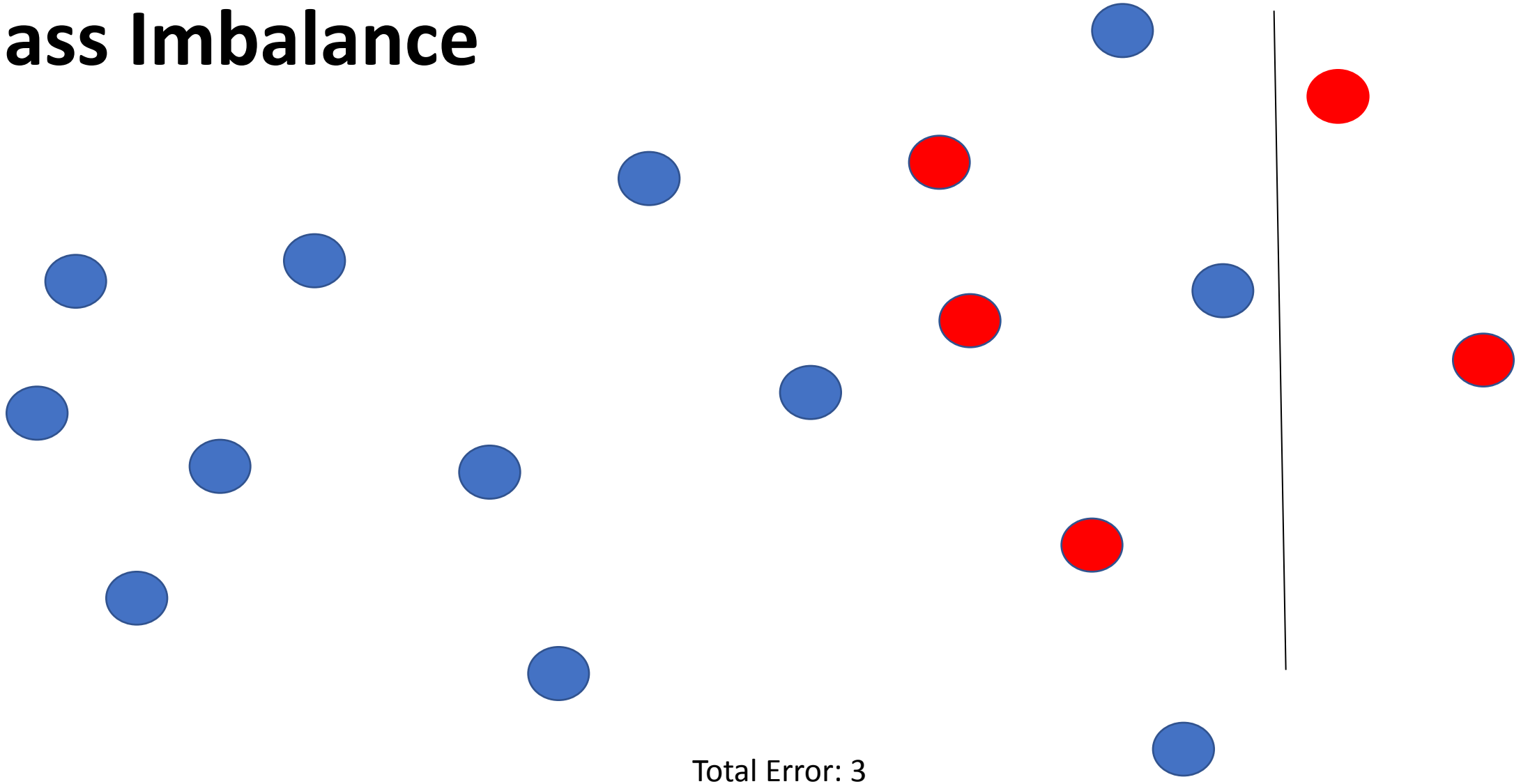
Mentimeter 5

Mentimeter 6

Class Imbalance

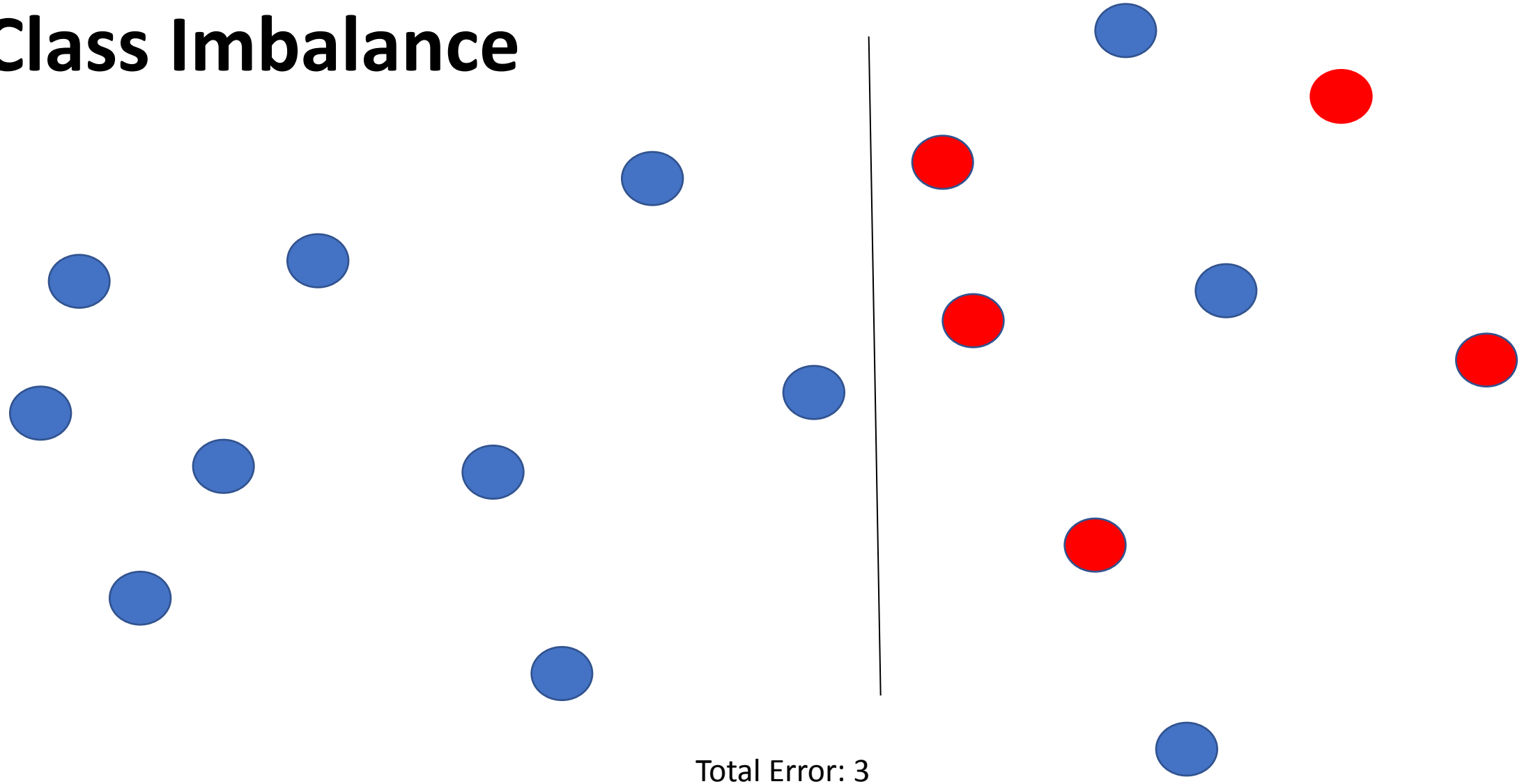


Class Imbalance



Total Error: 3

Class Imbalance



Total Error: 3

Class imbalance

- Dataset: blue: 12, red: 5
- In both cases, error = 3
- In case 1: blue error 0/12, red error 3/5
- In case 2: blue error 3/12, red error 0/5

Class imbalance

- Dataset: blue: 12, red: 5
- In both cases, error = 3
- In case 1: blue error 0/12, red error 3/5
- In case 2: blue error 3/12, red error 0/5

- Second classifier looks fairer to both classes
- But algorithm can pick either one!

Adjusting class weights

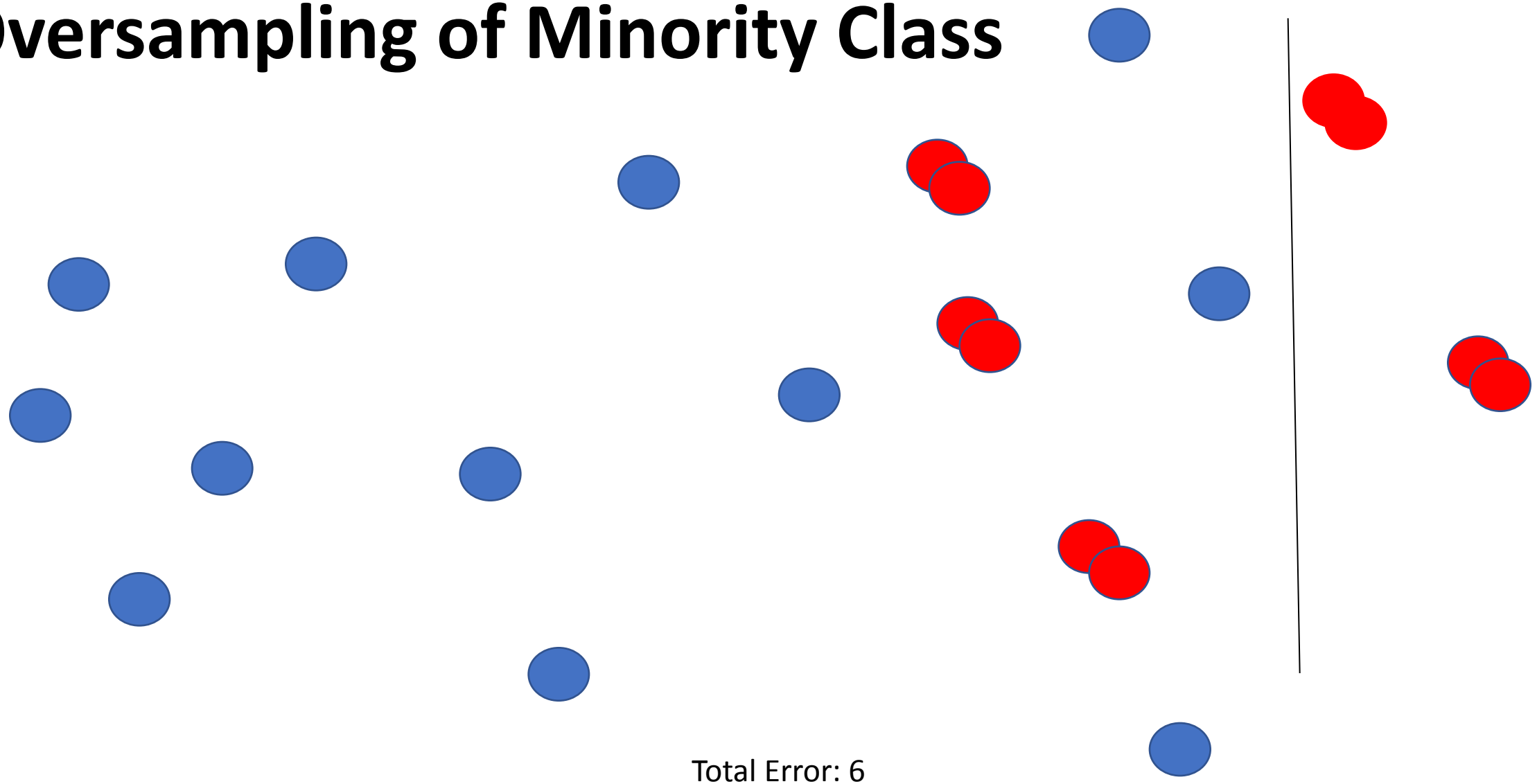
- Problem: majority and minority classes with examples of equal weight
- Solution: adjust the weights or frequencies

Adjusting class weights

- Problem: majority and minority classes with examples of equal weight
- Solution: adjust the weights or frequencies

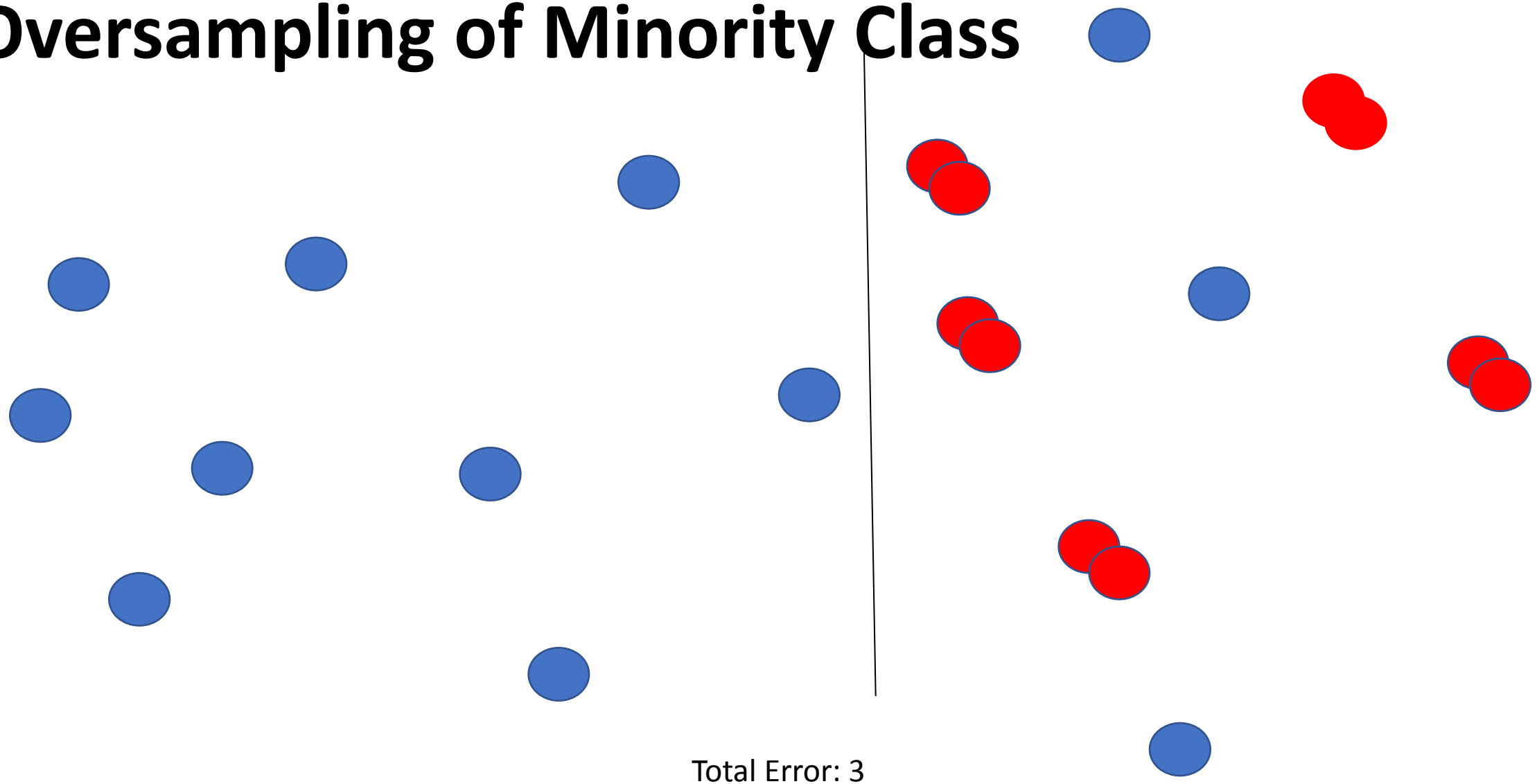
- Oversampling: introduce more examples from minority class
- Should be inexact copies of original samples (close in feature space)

Oversampling of Minority Class



Total Error: 6

Oversampling of Minority Class



Total Error: 3

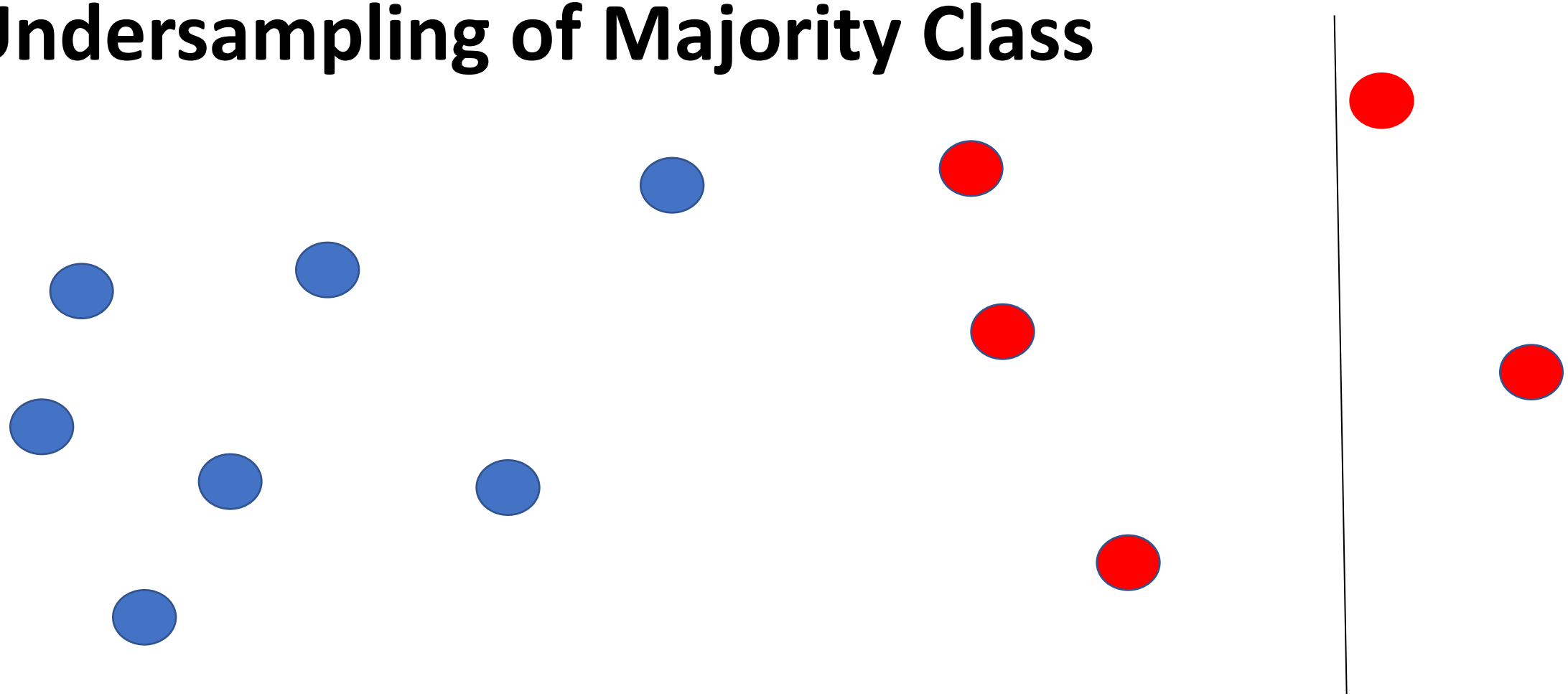
Adjusting class weights

- Problem: majority and minority classes with examples of equal weight
- Solution: adjust the weights or frequencies

- Oversampling: introduce more examples from minority class
- Should be inexact copies of original samples (close in feature space)

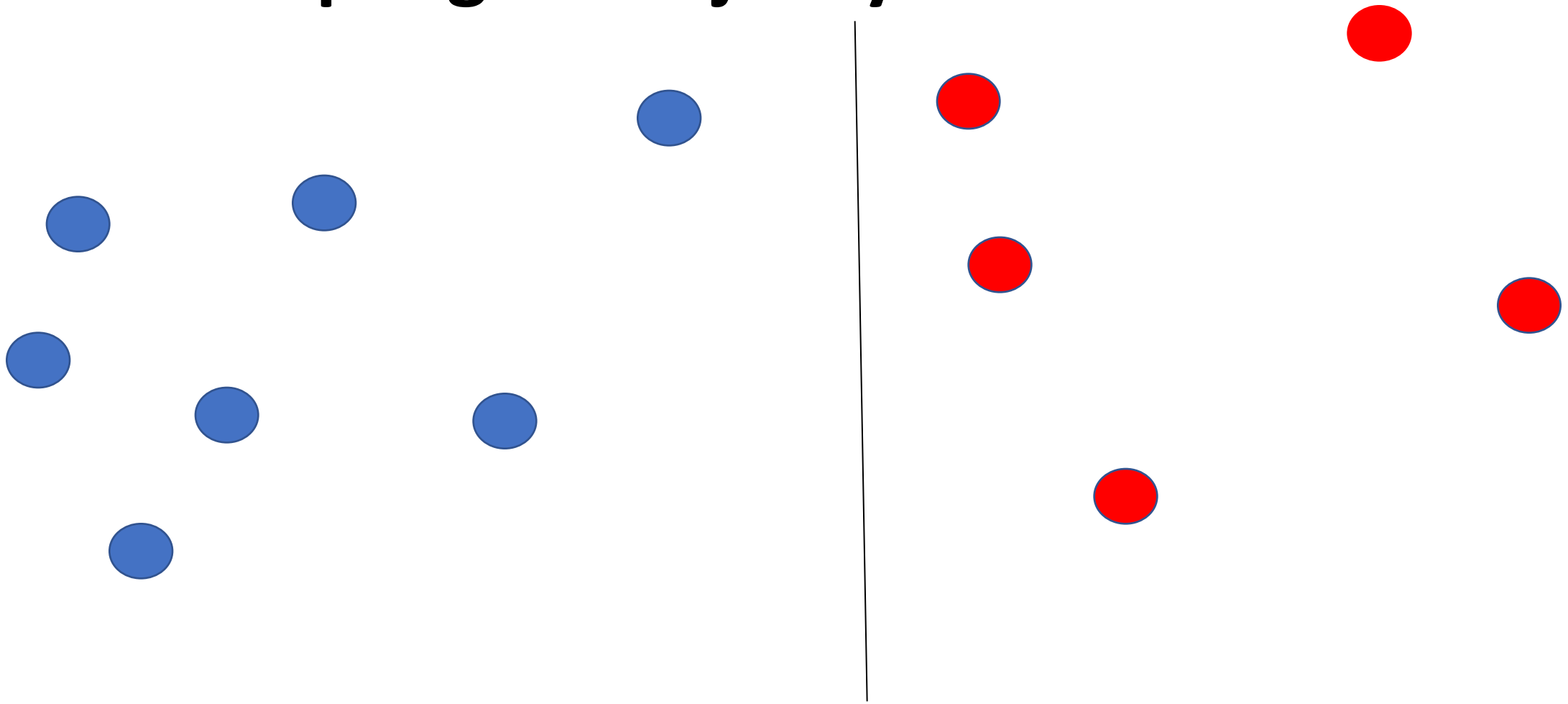
- Undersampling: ignore several examples from majority class
- Preferably, examples closer to decision boundary

Undersampling of Majority Class



Total Error: 3

Undersampling of Majority Class



Total Error: 0

Mentimeter 7

Mentimeter 8